Buddhini Ishani Database.docx

by WANNIGE BUDDHINI ISHANI

Submission date: 05-Sep-2026 03:48PM (UTC+0100)
Submission ID: 288333503
File name: 206445_WANNIGE_BUDDHINI_ISHANI_Buddhini_Ishani_Database_3211961_1855062873.docx (4.03M)
Word count: 6707
Character count: 37979

Assignment Cover Sheet





Cardiff Metropolitan University | Prifysgol Metropolitan Caerdydd

Higher Diploma in Computing & Software Engineering

- **Module :**

  CSE4204 - Database Design and Development

- **Student Name & No :**

  W.Buddhini Ishani - ST/HDCSE/CMU/57/01

Acknowledgement

I want to say thanks everyone who helped me during finish my Database design and development assignment based on SkyDeliver Drone Delivery Management System.

I would like to express my gratitude to my lecturer who provided us knowledge during this module and the advice and feedback given during the assignment. The knowledge gained from lectures and learning materials provided helped me to understand more about database such as data modelling, Entity Relationship Diagrams (ERD), relational database design, normalization, SQL, database security and testing.

I also want to thank the ICBT Campus for giving me the opportunity to learn and providing the learning facilities and resources which supported me to complete this assignment.

Last but not least, I want to thank my family and friends for the support and encouragement they gave me during the preparation of this assignment. They encouraged me and motivated so that I can complete the assignment on time

Executive Summary

The main objective of this assignment is to design and develop a database system for SkyDeliver drone delivery service. The database will store information about customers, deliveries, drones, hubs, assignments, maintenance, and battery charging.

The Relational Model and Document based NoSQL Model options will be discussed. The Relational Model will be chosen to design the database because the information stored in the database is highly structured. The Entity-Relationship Diagram will be developed and then transformed into a relational database by introducing primary and foreign keys. The tables will be normalized to the third normal form to eliminate data redundancies and dependencies. The developed database will be implemented in MS SQL Server by creating a database, tables, constraints, sample data, queries, stored procedures, and triggers. The testing process will be carried out to ensure that the database was implemented correctly. The database will also be backed up, secured, and recovered to maintain its integrity and availability.

Table of Contents

Acknowledgement I

Executive Summary II

List of Tables IV

List of Figures VI

Introduction VIII

TASK 1 - Data Model and Comparison 1

TASK 2 - Entity Relationship Diagram (ERD) 5

TASK 3 - I Relational Schema 7

TASK 3 - II Normalization to 3NF 9

TASK 4 - Database Implementation 26

TASK 5 - SQL queries 40

TASK 6 - Database Security, Backup and Recovery 45

TASK 7 - Test Plan and Test Cases 50

Conclusion 65

References 66

List of Tables

Table 1 : Relational Model 2
Table 2 : Document-Based NoSQL Model 3
Table 3 : Comparison of Relational Model vs NoSQL Model 4
Table 4 : UNF Customer 9
Table 5 : UNF DeliveryStatus 10
Table 6 : UNF DistributionHub 11
Table 7 : UNF Drone 11
Table 8 : UNF Delivery 11
Table 9 : UNF DroneAssignment 11
Table 10 : UNF Maintenance 11
Table 11 : UNF BatteryCharging 12
Table 12 : 1NF Customer 12
Table 13 : 1NF DeliveryStatus 13
Table 14 : 1NF DistributionHub 14
Table 15 : 1NF Drone 14
Table 16 : 1NF Delivery 14
Table 17 : 1NF DroneAssignment 15
Table 18 : 1NF Maintenance 15
Table 19 : 1NF BatteryCharging 15
Table 20 : 2NF Customer 17
Table 21 : 2NF Customer_Phone 17
Table 22 : 2NF 2. DeliveryStatus 18
Table 23 :2NF DistributionHub 19
Table 24 : 2NF Drone 19
Table 25 : 2NF Delivery 19
Table 26 : 2NF DroneAssignment 19
Table 27 : 2NF Maintenance 20
Table 28 : 2NF BatteryCharging 20
Table 29 : 3NF Customer 21
Table 30 : 3NF Customer_Phone 21
Table 31 : 3NF DeliveryStatus 22

Table 32 : 3NF DistributionHub 23
Table 33 : 3NF Drone 23
Table 34 : 3NF Delivery 24
Table 35 : 3NF DroneAssignment 24
Table 36 : 3NF Maintenance.....25
Table 37 : 3NF BatteryCharging.....25
Table 38 : Database Roles.....46
Table 39 : Backup and Recovery Schedule.....49
Table 40 : Test Resources.....53
Table 41 : Test Schedule 54
Table 42 : Test Cases.....64

List of Figures

Figure 1 : ERD for the SkyDeliver Drone Delivery Management System.....6

Figure 2 : Relational Schema.....8

Figure 3 : Create Database.....26

Figure 4 : Create Customer Table.....26

Figure 5 : Create Delivery Status Table.....27

Figure 6 : Create DistributionHub Table.....27

Figure 7 : Create Drone Table.....27

Figure 8 : Create Delivery Table.....28

Figure 9 : Create DroneAssignment Table.....28

Figure 10 : Create Maintenance Table.....29

Figure 11 : Create BatteryCharging Table.....29

Figure 12 : Insert Customer Records.....30

Figure 13 : Insert Delivery Status Records.....30

Figure 14 : Insert DistributionHub Records.....31

Figure 15 : Insert Drone Records.....31

Figure 16 : Insert Delivery Records.....32

Figure 17 : Insert DroneAssignment Records.....32

Figure 18 : Insert Maintenance Records.....33

Figure 19 : Insert BatteryCharging Records.....33

Figure 20 : Fetching Customer Data.....34

Figure 21 : Fetching DeliveryStatus Data.....34

Figure 22 : Fetching DistributionHub Data.....35

Figure 23 : Fetching Drone Data.....35

Figure 24 : Fetching Delivery Data.....36

Figure 25 : Fetching DroneAssignment Data.....36

Figure 26 : Fetching Maintenance Data.....37

Figure 27 : Fetching BatteryCharging Data.....37

Figure 28 : Stored Procedure.....38

Figure 29 : Test Stored Procedure.....38

Figure 30 : Create Trigger.....39

Figure 31 : Query 1.....40

Figure 32 : Query 2 41
Figure 33 : Query 3 42
Figure 34 : Query 4 43
Figure 35 : Query 5 44
Figure 37 : TC001 54
Figure 38 : TC002 55
Figure 39 : TC003 55
Figure 40 : TC004 56
Figure 41 : TC005 56
Figure 42 : TC006 57
Figure 43 : TC007 58
Figure 44 : TC008 58
Figure 45 : TC009 59
Figure 46 : TC 010 60
Figure 47 : TC 011 61
Figure 48 : TC 012 62
Figure 49 : TC 013 63
Figure 50 : TC 014 64

Introduction

The rapid growth of online shopping and on-demand delivery services has increased the need for fast, reliable, and efficient delivery solutions. Traditional delivery methods can face problems such as traffic congestion, delivery delays, and high operational costs. Drone-based delivery provides an alternative solution, especially for deliveries within urban areas.

SkyDeliver is a proposed drone-based delivery management system designed to manage and monitor parcel deliveries. Customers can place delivery requests through a mobile application by providing pickup and drop-off locations. The system can assign suitable drones based on availability, battery level, and payload capacity.

The database is a central part of SkyDeliver because it stores important information such as customer details, delivery requests, delivery status, drones, distribution hubs, assignments, maintenance, and battery charging records. It also supports the analysis of delivery performance and drone utilization.

This assignment focuses on designing and implementing a suitable database for SkyDeliver. It covers data model comparison, ERD development, relational schema design, normalization up to 3NF, SQL Server implementation, queries, security, backup and recovery, and database testing.

Overall, the proposed database is designed to provide a reliable, secure, and maintainable solution for supporting SkyDeliver's drone delivery operations.

TASK 1 - Data Model and Comparison

What is a Data Model ?

A data model is the description of how data is organised, stored, related, and managed in a database management system. It enables the designer to create databases with good quality, structure, and ease of maintenance.

The two broad classes of data models are the relational model and the NoSQL model. The relational model uses a table-based approach for storing data, whereas the NoSQL databases use various approaches such as document, key-value pair, column, and graph.

Relational Model

The Relational Model stores information in tables consisting of rows and columns. Usually, these tables represent some entities with the help of rows, columns, primary, and foreign keys. For instance, there can be a table for Customers and another one for Drones. Every table has a primary key that helps identify each entry uniquely, and foreign keys enable the connection between tables.

For example, SkyDeliver database can consist of several tables, such as Customer, Delivery, Drone, and DistributionHub. The first table's primary key can be CustomerID, and Delivery can have CustomerID as a foreign key. Moreover, the relational model typically uses Structured Query Language to design, create, manage, and manipulate databases. It has SQL constraints to ensure the data integrity of databases, such as primary and foreign keys, NOT NULL and UNIQUE constraints, etc.

Feature	Relational Model
Data structure	Tables, rows and columns
Schema	Predefined and structured
Relationships	Primary keys and foreign keys
Query language	SQL
Data integrity	Strong constraints and referential integrity
Best suited for	Structured and highly related data
SkyDeliver suitability	High

Table 1 : Relational Model

Document Based NoSQL Model

Instead of using conventional rows and columns, data is stored as documents in a document-based NoSQL database. JSON-like formats are frequently used to describe documents, and depending on the needs, they may have a variety of fields.

For instance, a SkyDeliver delivery document can have the delivery ID, client information, delivery address, drone details, and delivery status all in one document. Because various papers don't always have to have the same fields, this allows for flexibility.

Applications handling a lot of semi-structured or constantly changing data can benefit from document databases. One well-known example of a document-based NoSQL database is MongoDB.

Feature	Document-based NoSQL
Data structure	Documents / JSON-like objects
Schema	Flexible
Relationships	Usually embedding or references
Querying	Database-specific query languages
Data integrity	Often more application dependent

Best suited for	Flexible and rapidly changing data
SkyDeliver suitability	Possible, but less suitable for strict relationships

Table 2 : Document-Based NoSQL Model

Relational Model vs Document Based NoSQL

Feature	Relational Model	Document-Based NoSQL
Data storage	Tables containing rows and columns are used to store data.	Documents are used to store data, typically in a JSON-like format.
Schema	Uses a predefined and structured schema	Provides a flexible schema
Relationships	Uses primary keys and foreign keys to represent relationships	Relationships can be represented using references or embedded documents
Data integrity	Strong data integrity through constraints and relationships	Data integrity depends more on application and database design
Query language	SQL is commonly used	Query methods depend on the NoSQL database
Normalisation	Supports normalisation to reduce data redundancy	Data may be denormalised to improve performance
Transactions	Strong support for ACID transactions	Transaction support varies, although modern document databases support transactions
Scalability	Traditionally focuses on vertical scaling, although modern systems can also scale horizontally	Generally designed with horizontal scalability in mind
Flexibility	Changes to the structure may require schema modifications	Easier to store data with changing structures

Suitable data	Structured and highly related data	Semi-structured and rapidly changing data

Table 3 : Comparison of Relational Model vs NoSQL Model

Why Relational Databases Remain Widely Used

Relational databases are still widely used because they offer data consistency organized data handling, powerful SQL queries, support for transactions and reliable integrity rules. These features make them a solid choice for business systems where accurate and dependable transactions matter a lot.

NoSQL databases have become more popular as modern applications often deal with amounts of data constantly changing data formats and large numbers of users. Their flexible schema design and ability to scale horizontally make them useful for web apps, cloud platforms real-time systems and big data setups.

So the decision between these two types comes down to what the system needs. For the SkyDeliver delivery management system the relational model fits better since it handles structured data and many key relationships. However a document-based NoSQL database might also be considered for flexible data, like drone telemetry and extensive flight logs.

TASK 2 - Entity Relationship Diagram (ERD)

An Entity Relationship Diagram (ERD) can be referred to as a 'data model'. The purpose of it is to show the main objects within the system, known as entities, specify their keys, and identify their relationships with one another.

For the proposed SkyDeliver drone-based delivery management system, the ERD identifies customers, deliveries, drones, distribution hubs, drone assignments, maintenance activities, battery charging, and delivery statuses.

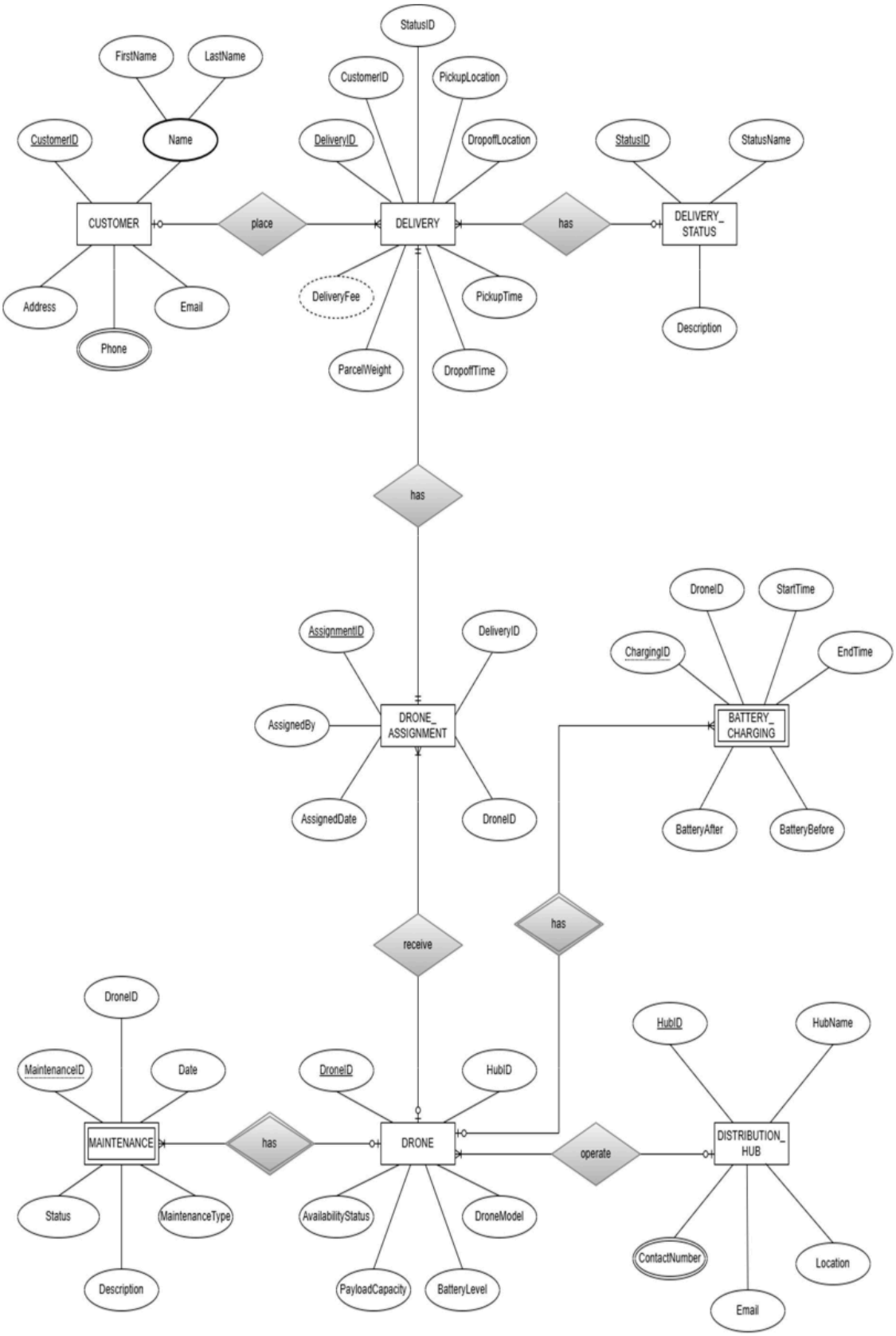


Figure 1 : ERD for the SkyDeliver Drone Delivery Management System

TASK 3 - I Relational Schema

A relational schema shows how a database is built with tables. In the SkyDeliver system the ERD that we created can be turned into tables by following the right mapping rules. Every strong entity becomes a table. The primary keys are. Foreign keys are added so the tables can talk to each other.

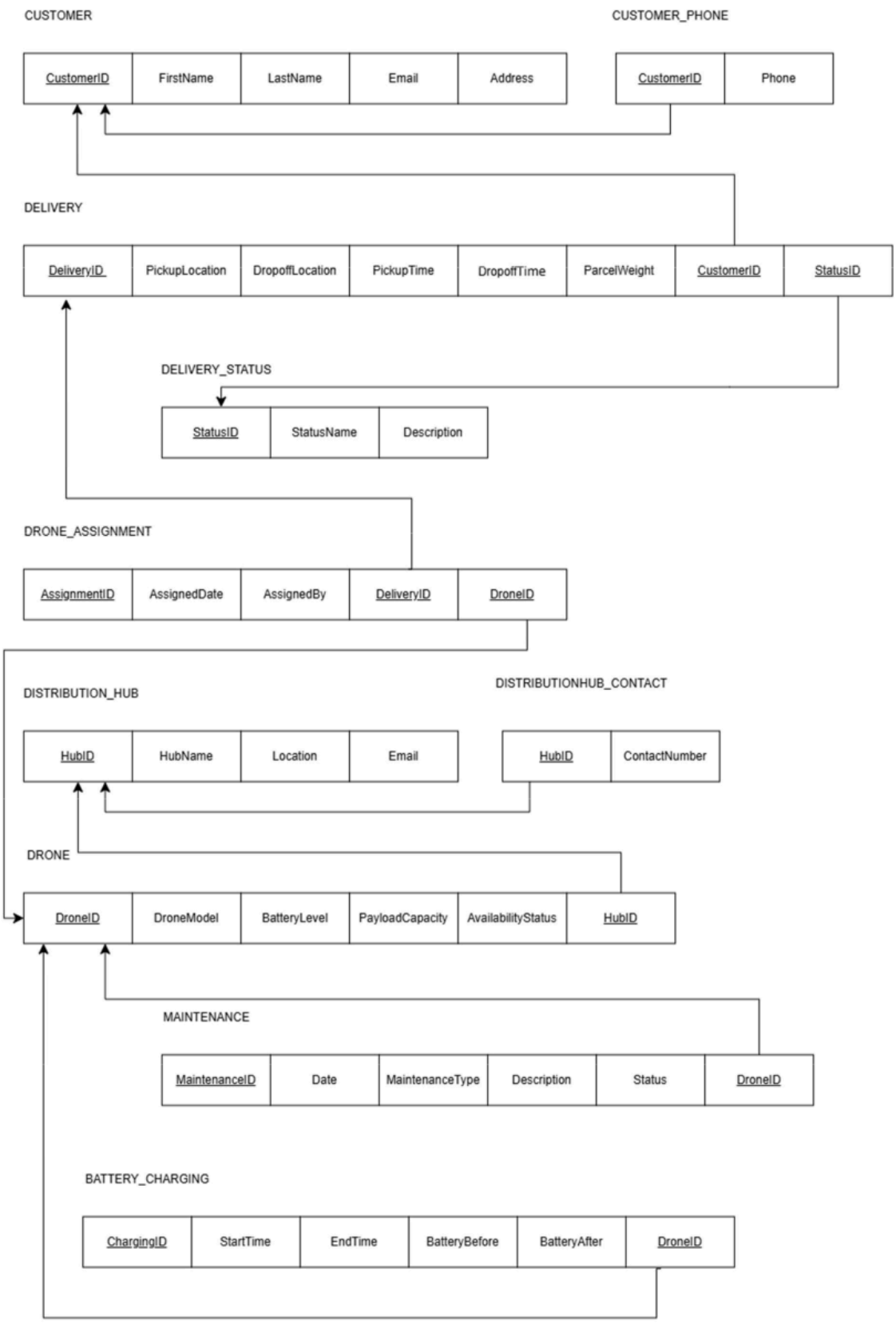


Figure 2 : Relational Schema

TASK 3 - II Normalization to 3NF

Normalization is the process of arranging data in a database. The goal is to make sure the data is organized properly. This helps to cut down on repeated data and make the data more reliable. The process includes using things, like Unnormalized Form, First Normal Form, Second Normal Form and Third Normal Form. The main purpose of normalization is to stop data from being repeated much. It also helps to avoid problems when adding, changing or removing data.

Unnormalized Form (UNF)

In the original Customer table, the Phone attribute can contain more than one phone number for a customer.

1. Customer Table

Customer ID	First Name	Last Name	Email	Address	Phone
1	Kamal	Perera	kamal.perera@gmail.com	Colombo 03	0771234567, 0705897801
2	Nimal	Fernando	nimal.fernando@gmail.com	Kandy	0712345678
3	Saman	Silva	saman.silva@gmail.com	Galle	0753456789, 0789567345
4	Amal	Perera	amal.perera@gmail.com	Negombo	0764567890, 0712538797
5	Kasun	Fernando	kasun.fernando@gmail.com	Gampaha	0785678901

Table 4 : UNF Customer

The Phone attribute contains multiple values, so the table is in UNF.

2. DeliveryStatus Table

StatusID	StatusName	Description
ST001	Pending	Delivery request has been received and is waiting for processing.
ST002	Confirmed	Delivery request has been confirmed by the system.
ST003	Assigned	A drone has been assigned to the delivery.
ST004	Pickup Scheduled	Pickup has been scheduled for the delivery.
ST005	Ready for Pickup	Parcel is ready to be collected from the pickup location.

Table 5 : UNF DeliveryStatus

There are no repeating or multivalued attributes.

3. DistributionHub Table

HubID	HubName	Location	ContactNumber	Email
HUB001	Colombo Central Hub	Colombo 01	0112345678	colombo@distributionhub.lk
HUB002	Kandy Distribution Hub	Kandy	0812345678	kandy@distributionhub.lk
HUB003	Galle Distribution Hub	Galle	0912345678	galle@distributionhub.lk
HUB004	Gampaha Distribution Hub	Gampaha	0332345678	gampaha@distributionhub.lk

HUB005	Negombo Distribution Hub	Negombo	0312345678	negombo@distributionhub.lk

Table 6 : UNF DistributionHub

There are no multivalued attributes.

4. Drone Table

DroneID	**HubID**	**DroneModel**	**BatteryLevel**	**Payload Capacity**	**AvailabilityStatus**

Table 7 : UNF Drone

5. Delivery Table

Delivery ID	**Customer ID**	**StatusID**	**Pickup Location**	**Dropoff Location**	**Pickup Time**	**Dropoff Time**

Table 8 : UNF Delivery

6. DroneAssignment Table

AssignmentID	**DeliveryID**	**DroneID**	**AssignedDate**	**AssignedBy**

Table 9 : UNF DroneAssignment

7. Maintenance Table

Maintenance ID	**DroneID**	**Date**	**MaintenanceType**	**Description**	**Status**

Table 10 : UNF Maintenance

8. BatteryCharging Table

ChargingID	DroneID	StartTime	EndTime	BatteryBefore	BatteryAfter

Table 11 : UNF BatteryCharging

First Normal Form (1NF)

Rule:

- Each column should contain atomic values.
- There should be no repeating groups or multiple values in one field.

1. Customer Table

The multiple phone numbers are separated into individual rows.

Customer ID	First Name	Last Name	Email	Address	Phone
1	Kamal	Perera	kamal.perera@gmail.com	Colombo 03	0771234567
1	Kamal	Perera	kamal.perera@gmail.com	Colombo 03	0705897801
2	Nimal	Fernando	nimal.fernando@gmail.com	Kandy	0712345678
3	Saman	Silva	saman.silva@gmail.com	Galle	0753456789
3	Saman	Silva	saman.silva@gmail.com	Galle	0789567345
4	Amal	Perera	amal.perera@gmail.com	Negombo	0764567890
4	Amal	Perera	amal.perera@gmail.com	Negombo	0712538797
5	Kasun	Fernando	kasun.fernando@gmail.com	Gampaha	0785678901

Table 12 : 1NF Customer

- Primary Key**:** CustomerID + Phone
- Now, each phone number is stored as a single value.
- Therefore, the table satisfies 1NF.

2. DeliveryStatus Table

StatusID	**StatusName**	**Description**
ST001	Pending	Delivery request has been received and is waiting for processing.
ST002	Confirmed	Delivery request has been confirmed by the system.
ST003	Assigned	A drone has been assigned to the delivery.
ST004	Pickup Scheduled	Pickup has been scheduled for the delivery.
ST005	Ready for Pickup	Parcel is ready to be collected from the pickup location.

Table 13 : 1NF DeliveryStatus

- Primary Key: StatusID
- All values are atomic, so the table is in 1NF.

3. DistributionHub Table

HubID	**HubName**	**Location**	**ContactNumber**	**Email**
HUB001	Colombo Central Hub	Colombo 01	0112345678	colombo@distributionhub.lk

HUB002	Kandy Distribution Hub	Kandy	0812345678	kandy@distributionhub.lk
HUB003	Galle Distribution Hub	Galle	0912345678	galle@distributionhub.lk
HUB004	Gampaha Distribution Hub	Gampaha	0332345678	gampaha@distributionhub.lk
HUB005	Negombo Distribution Hub	Negombo	0312345678	negombo@distributionhub.lk

Table 14 : 1NF DistributionHub

- Primary Key: HubID
- All values are atomic, so the table is in 1NF.

4. Drone Table

DroneID	**HubID**	**DroneModel**	**BatteryLevel**	**Payload Capacity**	**AvailabilityStatus**

Table 15 : 1NF Drone

- Primary Key: DroneID
- All values are atomic, so it is in 1NF.

5. Delivery Table

Delivery ID	**Customer ID**	**StatusID**	**Pickup Location**	**Dropoff Location**	**Pickup Time**	**Dropoff Time**

Table 16 : 1NF Delivery

- Primary Key: DeliveryID
- All values are atomic, therefore the table is in 1NF.

6. DroneAssignment Table

AssignmentID	DeliveryID	DroneID	AssignedDate	AssignedBy

Table 17 : 1NF DroneAssignment

- Primary Key: AssignmentID
- All values are atomic, therefore it is in 1NF.

7. Maintenance Table

Maintenance ID	DroneID	Date	Maintenance Type	Description	Status

Table 18 : 1NF Maintenance

- Primary Key: MaintenanceID
- All values are atomic, so it is in 1NF.

8. BatteryCharging Table

ChargingID	DroneID	StartTime	EndTime	BatteryBefore	BatteryAfter

Table 19 : 1NF BatteryCharging

- Primary Key: ChargingID
- All values are atomic, therefore it is in 1NF.

Second Normal Form (2NF)

Rule:

- Must be in 1NF.
- There should be no partial dependency.
- Non-key attributes must depend on the whole primary key.

1. Customer Table

- In the 1NF table, the primary key is: CustomerID + Phone
- However:
- CustomerID - FirstName, LastName, Email, Address
- This means customer details depend only on CustomerID, not on the whole composite key.
- Therefore, there is a partial dependency.
- To remove this dependency, the table is divided into two tables.

Customer ID	First Name	Last Name	Email	Address
1	Kamal	Perera	kamal.perera@gmail.com	Colombo 03
1	Kamal	Perera	kamal.perera@gmail.com	Colombo 03
2	Nimal	Fernando	nimal.fernando@gmail.com	Kandy
3	Saman	Silva	saman.silva@gmail.com	Galle
3	Saman	Silva	saman.silva@gmail.com	Galle
4	Amal	Perera	amal.perera@gmail.com	Negombo

4	Amal	Perera	amal.perera@gmail.com	Negombo
5	Kasun	Fernando	kasun.fernando@gmail.com	Gampaha

Table 20 : 2NF Customer

- **Primary Key:** CustomerID

Customer_Phone Table

CustomerID	**Phone**
1	0771234567
1	0705897801
2	0712345678
3	0753456789
3	0789567345
4	0764567890
4	0712538797
5	0785678901

Table 21 : 2NF Customer_Phone

- Primary Key: CustomerID + Phone
- Foreign Key: CustomerID

- The partial dependency has now been removed.
- Therefore, the tables are in 2NF.

2. DeliveryStatus Table

StatusID is a single-attribute primary key.

Therefore, there is no partial dependency.

StatusID	StatusName	Description
ST001	Pending	Delivery request has been received and is waiting for processing.
ST002	Confirmed	Delivery request has been confirmed by the system.
ST003	Assigned	A drone has been assigned to the delivery.
ST004	Pickup Scheduled	Pickup has been scheduled for the delivery.
ST005	Ready for Pickup	Parcel is ready to be collected from the pickup location.

Table 22 : 2NF 2. DeliveryStatus

The table is in 2NF.

3. DistributionHub Table

HubID	HubName	Location	ContactNumber	Email
HUB001	Colombo Central Hub	Colombo 01	0112345678	colombo@distributionhub.lk
HUB002	Kandy Distribution Hub	Kandy	0812345678	kandy@distributionhub.lk
HUB003	Galle Distribution Hub	Galle	0912345678	galle@distributionhub.lk
HUB004	Gampaha Distribution Hub	Gampaha	0332345678	gampaha@distributionhub.lk

HUB005	Negombo Distribution Hub	Negombo	0312345678	negombo@distributionhub.lk

Table 23 :2NF DistributionHub

Since HubID is a single primary key, there is no partial dependency.

Therefore, it is in 2NF.

4. Drone Table

DroneID	**HubID**	**DroneModel**	**BatteryLevel**	**Payload Capacity**	**AvailabilityStatus**

Table 24 : 2NF Drone

DroneID is a single primary key, so there is no partial dependency.

Therefore, it is in 2NF.

5. Delivery Table

Delivery ID	**Customer ID**	**StatusID**	**Pickup Location**	**Dropoff Location**	**Pickup Time**	**Dropoff Time**

Table 25 : 2NF Delivery

DeliveryID is a single primary key, so there is no partial dependency.

Therefore, the table is in 2NF.

6. DroneAssignment Table

AssignmentID	**DeliveryID**	**DroneID**	**AssignedDate**	**AssignedBy**

Table 26 : 2NF DroneAssignment

There is no partial dependency because AssignmentID is a single primary key.

Therefore, it is in 2NF.

7. Maintenance Table

Maintenance ID	DroneID	Date	Maintenance Type	Description	Status

Table 27 : 2NF Maintenance

There is no partial dependency.

Therefore, it is in 2NF.

8. BatteryCharging Table

ChargingID	DroneID	StartTime	EndTime	BatteryBefore	BatteryAfter

Table 28 : 2NF BatteryCharging

There is no partial dependency because ChargingID is a single primary key.

Therefore, it is in 2NF.

Third Normal Form (3NF)

Rule:

- Must be in 2NF.
- There should be no transitive dependency.
- Non-key attributes should not depend on other non-key attributes.

For the Customer table:

All the non-key attributes are dependent upon CustomerID.

There is no such dependency as:

CustomerID - Non-key attribute - Another non-key attribute

Hence, there is no transitive dependency.

1. Customer Table

CustomerID	FirstName	LastName	Email	Address
1	Kamal	Perera	kamal.perera@gmail.com	Colombo 03
2	Nimal	Fernando	nimal.fernando@gmail.com	Kandy
3	Saman	Silva	saman.silva@gmail.com	Galle
4	Amal	Perera	amal.perera@gmail.com	Negombo
5	Kasun	Fernando	kasun.fernando@gmail.com	Gampaha

Table 29 : 3NF Customer

Customer_Phone Table

CustomerID	Phone
1	0771234567
1	0705897801
2	0712345678
3	0753456789
3	0789567345
4	0764567890
4	0712538797
5	0785678901

Table 30 : 3NF Customer_Phone

Therefore, the Customer data is in 3NF.

2. DeliveryStatus Table

StatusID	StatusName	Description
ST001	Pending	Delivery request has been received and is waiting for processing.
ST002	Confirmed	Delivery request has been confirmed by the system.
ST003	Assigned	A drone has been assigned to the delivery.
ST004	Pickup Scheduled	Pickup has been scheduled for the delivery.
ST005	Ready for Pickup	Parcel is ready to be collected from the pickup location.

Table 31 : 3NF DeliveryStatus

There is no transitive dependency because StatusName and Description depend directly on StatusID.

Therefore, the table is in 3NF.

3. DistributionHub Table

HubID	HubName	Location	ContactNumber	Email
HUB001	Colombo Central Hub	Colombo 01	0112345678	colombo@distributionhub.lk
HUB002	Kandy Distribution Hub	Kandy	0812345678	kandy@distributionhub.lk
HUB003	Galle Distribution Hub	Galle	0912345678	galle@distributionhub.lk

HUB004	Gampaha Distribution Hub	Gampaha	0332345678	gampaha@distributionhub.lk
HUB005	Negombo Distribution Hub	Negombo	0312345678	negombo@distributionhub.lk

Table 32 : 3NF DistributionHub

All non-key attributes depend directly on HubID.

There is no transitive dependency.

4. Drone Table

DroneID	**HubID**	**DroneModel**	**BatteryLevel**	**Payload Capacity**	**AvailabilityStatus**
DRN001	HUB001	Sky-X1	95.50	9.00	Available
DRN002	HUB002	Sky-X2	88.20	9.00	Available
DRN003	HUB003	Sky-Pro1	76.80	7.50	In Use
DRN004	HUB005	Sky-Pro2	92.40	2.70	Available
DRN005	HUB005	Sky-X3	65.30	9.00	Maintenance

Table 33 : 3NF Drone

All non-key attributes depend directly on DroneID

5. Delivery Table

Delivery ID	**Customer ID**	**Status ID**	**Pickup Location**	**Dropoff Location**	**Pickup Time**	**Dropoff Time**
1	1	ST006	Colombo 03	Colombo 01	2026-08-01 09:00	2026-08-01 10:30
2	2	ST011	Kandy	Peradeniya	2026-08-02 10:00	NULL

3	3	ST007	Galle	Unawatuna	2026-08-03 09:30	2026-08-03 11:20
4	4	ST011	Negombo	Katunayake	2026-08-04 11:00	NULL
5	5	ST007	Gampaha	Kadawatha	2026-08-05 09:00	NULL

Table 34 : 3NF Delivery

6. DroneAssignment Table

AssignmentID	**DeliveryID**	**DroneID**	**AssignedDate**	**AssignedBy**
1	DL001	DRN001	2026-08-01	Admin
2	DL002	DRN003	2026-08-02	Admin
3	DL003	DRN006	2026-08-03	System
4	DL004	DRN009	2026-08-04	Admin
5	DL005	DRN007	2026-08-05	Manager

Table 35 : 3NF DroneAssignment

25 There is no transitive dependency.

All non-key attributes depend directly on DeliveryID, and there is no transitive dependency.

7. Maintenance Table

Maintenance ID	**DroneID**	**Date**	**Maintenance Type**	**Description**	**Status**
1	DRN001	2026-07-01	Routine Inspection	Battery and motor inspection	Completed
2	DRN002	2026-07-02	Battery Check	Battery performance checked	Completed

3	DRN003	2026-07-03	Motor Service	Motor performance checked	Completed
4	DRN004	2026-07-04	Software Update	Flight software updated	Completed
5	DRN005	2026-07-05	Motor Repair	Motor maintenance required	In Progress

Table 36 : 3NF Maintenance

All non-key attributes depend directly on MaintenanceID, so there is no transitive dependency.

8. BatteryCharging Table

ChargingID	**StartTime**	**EndTime**	**BatteryBefore**	**BatteryAfter**	**DroneID**
CHG001	2026-08-01 06:00	2026-08-01 07:30	35.00	95.00	DRN001
CHG002	2026-08-02 06:30	2026-08-02 08:00	40.00	96.00	DRN002
CHG003	2026-08-03 07:00	2026-08-03 08:30	28.00	90.00	DRN003
CHG004	2026-08-04 06:00	2026-08-04 07:20	32.00	94.00	DRN004
CHG005	2026-08-05 05:30	2026-08-05 07:00	25.00	92.00	DRN005

Table 37 : 3NF BatteryCharging

There is no transitive dependency because all non-key attributes depend directly on ChargingID.

TASK 4 - Database Implementation

IshaniB.sql

```
IshaniB.sql - ST-V...ter (student (54))*
-- SKY DELIVER DATABASE --

-- CREATE DATABASE --

CREATE DATABASE SkyDeliverDB;

Messages
Command(s) completed successfully.
```

Figure 3 : Create Database

```
SkyDeliverDB | Execute
IshaniB.sql - ST-V...rDB (student (54))*
-- CREATE TABLES --

CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15),
    Address VARCHAR(200)
);

Messages
Command(s) completed successfully.
```

Figure 4 : Create Customer Table

```
CREATE TABLE DeliveryStatus (
    StatusID VARCHAR(10) PRIMARY KEY,
    StatusName VARCHAR(50) NOT NULL,
    Description VARCHAR(200)
);
```

```
Messages
Command(s) completed successfully.
```

Figure 5 : Create Delivery Status Table

```
CREATE TABLE DistributionHub (
    HubID VARCHAR(10) PRIMARY KEY,
    HubName VARCHAR(100) NOT NULL,
    Location VARCHAR(150) NOT NULL,
    ContactNumber VARCHAR(15),
    Email VARCHAR(50)
);
```

```
Messages
Command(s) completed successfully.
```

Figure 6 : Create DistributionHub Table

```
CREATE TABLE Drone (
    DroneID VARCHAR(10) PRIMARY KEY,
    HubID VARCHAR(10) NOT NULL,
    DroneModel VARCHAR(50) NOT NULL,
    BatteryLevel DECIMAL(5,2),
    PayloadCapacity DECIMAL(6,2),
    AvailabilityStatus VARCHAR(30),

    FOREIGN KEY (HubID) REFERENCES DistributionHub(HubID)
);
```

```
Messages
Command(s) completed successfully.
```

Figure 7 : Create Drone Table

```
CREATE TABLE Delivery (
    DeliveryID VARCHAR(10) PRIMARY KEY,
    CustomerID INT NOT NULL,
    StatusID VARCHAR(10) NOT NULL,
    PickupLocation VARCHAR(150) NOT NULL,
    DropoffLocation VARCHAR(150) NOT NULL,
    PickupTime DATETIME NULL,
    DropoffTime  DATETIME NULL,
    ParcelWeight DECIMAL(6,2),

   FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
   FOREIGN KEY (StatusID) REFERENCES DeliveryStatus(StatusID)
);
```

Messages

Command(s) completed successfully.

Figure 8 : Create Delivery Table

```
CREATE TABLE DroneAssignment (
    AssignmentID INT PRIMARY KEY,
    DeliveryID VARCHAR(10)  NOT NULL,
    DroneID VARCHAR(10) NOT NULL,
    AssignmentDate DATE NOT NULL,
    AssignedBy VARCHAR(100),

    FOREIGN KEY (DeliveryID) REFERENCES Delivery(DeliveryID),
    FOREIGN KEY (DroneID) REFERENCES Drone(DroneID)
);
```

Messages

Command(s) completed successfully.

Figure 9 : Create DroneAssignment Table

```
CREATE TABLE Maintenance (
    MaintenanceID INT PRIMARY KEY,
    DroneID VARCHAR(10)  NOT NULL,
    MaintenanceDate DATE NOT NULL,
    MaintenanceType VARCHAR(50),
    Description VARCHAR(200),
    MaintenanceStatus VARCHAR(30),

    FOREIGN KEY (DroneID) REFERENCES Drone(DroneID)
);
```

```
Messages
Command(s) completed successfully.
```

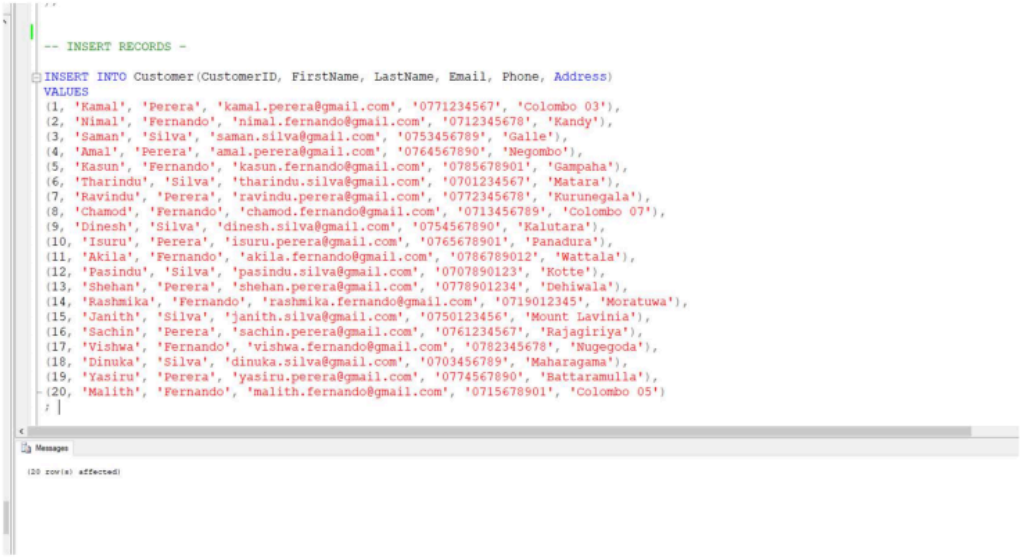
Figure 10 : Create Maintenance Table

```
CREATE TABLE BatteryCharging (
    ChargingID VARCHAR(10) PRIMARY KEY,
    DroneID VARCHAR(10) NOT NULL,
    StartTime DATETIME NOT NULL,
    EndTime DATETIME,
    BatteryBefore DECIMAL(5,2),
    BatteryAfter DECIMAL(5,2),

    FOREIGN KEY (DroneID) REFERENCES Drone(DroneID)
);
```

```
Messages
Command(s) completed successfully.
```

Figure 11 : Create BatteryCharging Table



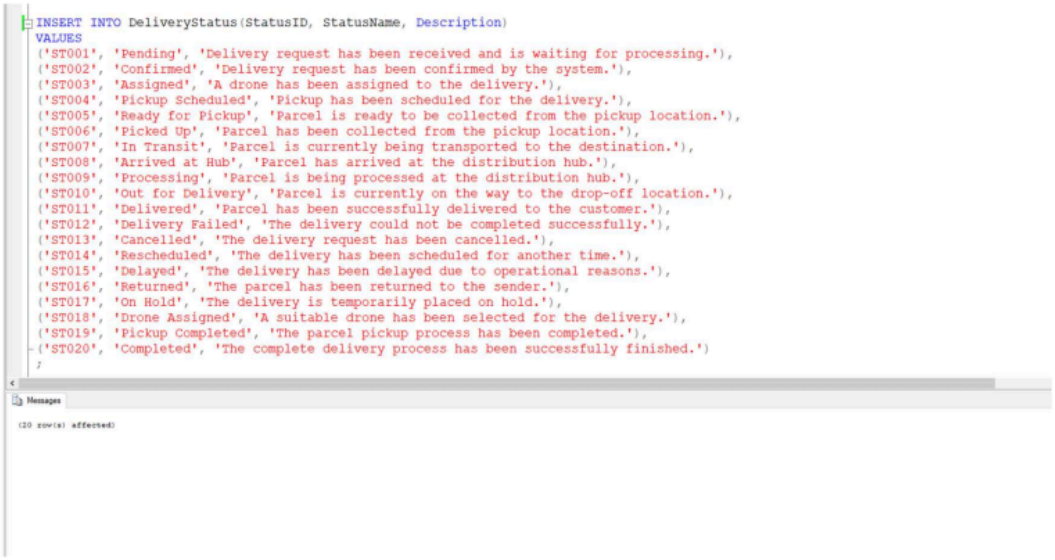
```
-- INSERT RECORDS -

INSERT INTO Customer(CustomerID, FirstName, LastName, Email, Phone, Address)
VALUES
(1, 'Kamal', 'Perera', 'kamal.perera@gmail.com', '0771234567', 'Colombo 03'),
(2, 'Nimal', 'Fernando', 'nimal.fernando@gmail.com', '0712345678', 'Kandy'),
(3, 'Saman', 'Silva', 'saman.silva@gmail.com', '0753456789', 'Galle'),
(4, 'Amal', 'Perera', 'amal.perera@gmail.com', '0764567890', 'Negombo'),
(5, 'Kasun', 'Fernando', 'kasun.fernando@gmail.com', '0785678901', 'Gampaha'),
(6, 'Tharindu', 'Silva', 'tharindu.silva@gmail.com', '0701234567', 'Matara'),
(7, 'Ravindu', 'Perera', 'ravindu.perera@gmail.com', '0772345678', 'Kurunegala'),
(8, 'Chamod', 'Fernando', 'chamod.fernando@gmail.com', '0713456789', 'Colombo 07'),
(9, 'Dinesh', 'Silva', 'dinesh.silva@gmail.com', '0754567890', 'Kalutara'),
(10, 'Isuru', 'Perera', 'isuru.perera@gmail.com', '0765678901', 'Panadura'),
(11, 'Akila', 'Fernando', 'akila.fernando@gmail.com', '0786789012', 'Wattala'),
(12, 'Pasindu', 'Silva', 'pasindu.silva@gmail.com', '0707890123', 'Kotte'),
(13, 'Shehan', 'Perera', 'shehan.perera@gmail.com', '0778901234', 'Dehiwala'),
(14, 'Rashmika', 'Fernando', 'rashmika.fernando@gmail.com', '0719012345', 'Moratuwa'),
(15, 'Janith', 'Silva', 'janith.silva@gmail.com', '0750123456', 'Mount Lavinia'),
(16, 'Sachin', 'Perera', 'sachin.perera@gmail.com', '0761234567', 'Rajagiriya'),
(17, 'Vishwa', 'Fernando', 'vishwa.fernando@gmail.com', '0782345678', 'Nugegoda'),
(18, 'Dinuka', 'Silva', 'dinuka.silva@gmail.com', '0703456789', 'Maharagama'),
(19, 'Yasiru', 'Perera', 'yasiru.perera@gmail.com', '0774567890', 'Battaramulla'),
(20, 'Malith', 'Fernando', 'malith.fernando@gmail.com', '0715678901', 'Colombo 05')
;
```

Messages

(20 row(s) affected)

Figure 12 : Insert Customer Records



```
INSERT INTO DeliveryStatus(StatusID, StatusName, Description)
VALUES
('ST001', 'Pending', 'Delivery request has been received and is waiting for processing.'),
('ST002', 'Confirmed', 'Delivery request has been confirmed by the system.'),
('ST003', 'Assigned', 'A drone has been assigned to the delivery.'),
('ST004', 'Pickup Scheduled', 'Pickup has been scheduled for the delivery.'),
('ST005', 'Ready for Pickup', 'Parcel is ready to be collected from the pickup location.'),
('ST006', 'Picked Up', 'Parcel has been collected from the pickup location.'),
('ST007', 'In Transit', 'Parcel is currently being transported to the destination.'),
('ST008', 'Arrived at Hub', 'Parcel has arrived at the distribution hub.'),
('ST009', 'Processing', 'Parcel is being processed at the distribution hub.'),
('ST010', 'Out for Delivery', 'Parcel is currently on the way to the drop-off location.'),
('ST011', 'Delivered', 'Parcel has been successfully delivered to the customer.'),
('ST012', 'Delivery Failed', 'The delivery could not be completed successfully.'),
('ST013', 'Cancelled', 'The delivery request has been cancelled.'),
('ST014', 'Rescheduled', 'The delivery has been scheduled for another time.'),
('ST015', 'Delayed', 'The delivery has been delayed due to operational reasons.'),
('ST016', 'Returned', 'The parcel has been returned to the sender.'),
('ST017', 'On Hold', 'The delivery is temporarily placed on hold.'),
('ST018', 'Drone Assigned', 'A suitable drone has been selected for the delivery.'),
('ST019', 'Pickup Completed', 'The parcel pickup process has been completed.'),
('ST020', 'Completed', 'The complete delivery process has been successfully finished.')
;
```

Messages

(20 row(s) affected)

Figure 13 : Insert Delivery Status Records

```
INSERT INTO DistributionHub(HubID, HubName, Location, ContactNumber, Email)
VALUES
('HUB001', 'Colombo Central Hub', 'Colombo 01', '0112345678', 'colombo@distributionhub.lk'),
('HUB002', 'Kandy Distribution Hub', 'Kandy', '0812345678', 'kandy@distributionhub.lk'),
('HUB003', 'Galle Distribution Hub', 'Galle', '0912345678', 'galle@distributionhub.lk'),
('HUB004', 'Gampaha Distribution Hub', 'Gampaha', '0332345678', 'gampaha@distributionhub.lk'),
('HUB005', 'Negombo Distribution Hub', 'Negombo', '0312345678', 'negombo@distributionhub.lk'),
('HUB006', 'Matara Distribution Hub', 'Matara', '0412345678', 'matara@distributionhub.lk'),
('HUB007', 'Kurunegala Distribution Hub', 'Kurunegala', '0372345678', 'kurunegala@distributionhub.lk'),
('HUB008', 'Kalutara Distribution Hub', 'Kalutara', '0342345678', 'kalutara@distributionhub.lk'),
('HUB009', 'Ratnapura Distribution Hub', 'Ratnapura', '0452345678', 'ratnapura@distributionhub.lk'),
('HUB010', 'Anuradhapura Distribution Hub', 'Anuradhapura', '0252345678', 'anuradhapura@distributionhub.lk'),
('HUB011', 'Jaffna Distribution Hub', 'Jaffna', '0212345678', 'jaffna@distributionhub.lk'),
('HUB012', 'Batticaloa Distribution Hub', 'Batticaloa', '0652345678', 'batticaloa@distributionhub.lk'),
('HUB013', 'Trincomalee Distribution Hub', 'Trincomalee', '0262345678', 'trincomalee@distributionhub.lk'),
('HUB014', 'Badulla Distribution Hub', 'Badulla', '0552345678', 'badulla@distributionhub.lk'),
('HUB015', 'Nuwara Eliya Distribution Hub', 'Nuwara Eliya', '0522345678', 'nuwaraeliya@distributionhub.lk'),
('HUB016', 'Avissawella Distribution Hub', 'Avissawella', '0362345678', 'avissawella@distributionhub.lk'),
('HUB017', 'Panadura Distribution Hub', 'Panadura', '0382345678', 'panadura@distributionhub.lk'),
('HUB018', 'Moratuwa Distribution Hub', 'Moratuwa', '0113456789', 'moratuwa@distributionhub.lk'),
('HUB019', 'Wattala Distribution Hub', 'Wattala', '0114567890', 'wattala@distributionhub.lk'),
('HUB020', 'Maharagama Distribution Hub', 'Maharagama', '0115678901', 'maharagama@distributionhub.lk')
;
```

Messages

(20 row(s) affected)

12

Figure 14 : Insert DistributionHub Records

```
;
INSERT INTO Drone(DroneID, HubID, DroneModel, BatteryLevel, PayloadCapacity, AvailabilityStatus)
VALUES
    ('DRN001', 'HUB001', 'Sky-X1', 95.50, 9.00, 'Available'),
    ('DRN002', 'HUB002', 'Sky-X2', 88.20, 9.00, 'Available'),
    ('DRN003', 'HUB003', 'Sky-Pro1', 76.80, 7.50, 'In Use'),
    ('DRN004', 'HUB005', 'Sky-Pro2', 92.40, 2.70, 'Available'),
    ('DRN005', 'HUB005', 'Sky-X3', 65.30, 9.00, 'Maintenance'),
    ('DRN006', 'HUB003', 'Sky-Max1', 81.60, 7.50, 'Available'),
    ('DRN007', 'HUB004', 'Sky-Max2', 97.10, 9.00, 'Available'),
    ('DRN008', 'HUB014', 'Sky-X4', 72.50, 2.70, 'In Use'),
    ('DRN009', 'HUB008', 'Sky-Pro3', 84.90, 9.00, 'Available'),
    ('DRN010', 'HUB007', 'Sky-Max3', 59.40, 7.50, 'Charging'),
    ('DRN011', 'HUB008', 'Sky-X5', 90.30, 9.00, 'Available'),
    ('DRN012', 'HUB009', 'Sky-M30 Series', 68.70, 2.70, 'In Use'),
    ('DRN013', 'HUB010', 'Sky-Pro4', 93.80, 9.00, 'Available'),
    ('DRN014', 'HUB012', 'Sky-Max4', 79.20, 7.50, 'Available'),
    ('DRN015', 'HUB014', 'Sky-X1', 61.50, 9.00, 'Maintenance'),
    ('DRN016', 'HUB001', 'Sky-M30 Series', 86.40, 2.70, 'Available'),
    ('DRN017', 'HUB002', 'Sky-Max5', 74.60, 9.00, 'In Use'),
    ('DRN018', 'HUB018', 'Sky-Max5', 96.20, 7.50, 'Available'),
    ('DRN019', 'HUB020', 'Sky-X7', 70.80, 9.00, 'Charging'),
    ('DRN020', 'HUB005', 'Sky-Pro6', 89.50, 2.70, 'Available')
;
```

Messages

(20 row(s) affected)

Figure 15 : Insert Drone Records

```
INSERT INTO Delivery(DeliveryID, CustomerID, StatusID, PickupLocation, DropoffLocation, PickupTime, DropoffTime, ParcelWeight)
VALUES
('DL001', 1, 'ST011', 'Colombo 03', 'Colombo 01', '2026-08-01 09:00', '2026-08-01 10:30', 2.50),
('DL002', 2, 'ST007', 'Kandy', 'Peradeniya', '2026-08-02 10:00', NULL, 3.20),
('DL003', 3, 'ST011', 'Galle', 'Unawatuna', '2026-08-03 09:30', '2026-08-03 11:20', 1.80),
('DL004', 4, 'ST006', 'Negombo', 'Katunayake', '2026-08-04 11:00', NULL, 4.50),
('DL005', 5, 'ST007', 'Gampaha', 'Kadawatha', '2026-08-05 09:00', NULL, 5.00),
('DL006', 6, 'ST011', 'Matara', 'Weligama', '2026-08-06 08:30', '2026-08-06 10:10', 2.20),
('DL007', 7, 'ST010', 'Kurunegala', 'Kuliyapitiya', '2026-08-07 10:00', NULL, 6.00),
('DL008', 8, 'ST011', 'Colombo 07', 'Borella', '2026-08-08 08:45', '2026-08-08 10:00', 1.50),
('DL009', 9, 'ST007', 'Kalutara', 'Beruwala', '2026-08-09 10:30', NULL, 3.70),
('DL010', 10, 'ST011', 'Panadura', 'Wadduwa', '2026-08-10 08:00', '2026-08-10 09:15', 2.00),
('DL011', 11, 'ST003', 'Wattala', 'Colombo 04', NULL, NULL, 4.20),
('DL012', 12, 'ST004', 'Kotte', 'Nugegoda', NULL, NULL, 2.80),
('DL013', 13, 'ST011', 'Dehiwala', 'Wellawatte', '2026-08-13 08:45', '2026-08-13 10:20', 3.00),
('DL014', 14, 'ST007', 'Moratuwa', 'Ratmalana', '2026-08-14 10:00', NULL, 4.80),
('DL015', 15, 'ST011', 'Mount Lavinia', 'Dehiwala', '2026-08-15 08:30', '2026-08-15 09:45', 1.20),
('DL016', 16, 'ST005', 'Rajagiriya', 'Battaramulla', NULL, NULL, 3.50),
('DL017', 17, 'ST010', 'Nugegoda', 'Maharagama', '2026-08-17 09:15', NULL, 5.50),
('DL018', 18, 'ST011', 'Maharagama', 'Kottawa', '2026-08-18 08:15', '2026-08-18 09:40', 2.40),
('DL019', 19, 'ST002', 'Battaramulla', 'Kotte', NULL, NULL, 4.00),
('DL020', 20, 'ST003', 'Colombo 05', 'Colombo 02', NULL, NULL, 3.30)
;
```

Messages

```
(20 row(s) affected)
```

Figure 16 : Insert Delivery Records

```
INSERT INTO DroneAssignment (AssignmentID, DeliveryID, DroneID, AssignmentDate, AssignedBy)
VALUES
(1, 'DL001', 'DRN001', '2026-08-01', 'Admin'),
(2, 'DL002', 'DRN003', '2026-08-02', 'Admin'),
(3, 'DL003', 'DRN006', '2026-08-03', 'System'),
(4, 'DL004', 'DRN009', '2026-08-04', 'Admin'),
(5, 'DL005', 'DRN007', '2026-08-05', 'Manager'),
(6, 'DL006', 'DRN018', '2026-08-06', 'System'),
(7, 'DL007', 'DRN008', '2026-08-07', 'Manager'),
(8, 'DL008', 'DRN011', '2026-08-08', 'Admin'),
(9, 'DL009', 'DRN013', '2026-08-09', 'System'),
(10, 'DL010', 'DRN016', '2026-08-10', 'Admin'),
(11, 'DL011', 'DRN002', '2026-08-11', 'Manager'),
(12, 'DL012', 'DRN004', '2026-08-12', 'System'),
(13, 'DL013', 'DRN014', '2026-08-13', 'Admin'),
(14, 'DL014', 'DRN017', '2026-08-14', 'Manager'),
(15, 'DL015', 'DRN020', '2026-08-15', 'System'),
(16, 'DL016', 'DRN001', '2026-08-16', 'Admin'),
(17, 'DL017', 'DRN003', '2026-08-17', 'Manager'),
(18, 'DL018', 'DRN006', '2026-08-18', 'System'),
(19, 'DL019', 'DRN009', '2026-08-19', 'Admin'),
(20, 'DL020', 'DRN012', '2026-08-19', 'Manager')
;
```

Messages

```
(20 row(s) affected)
```

Figure 17 : Insert DroneAssignment Records

```
INSERT INTO Maintenance
(MaintenanceID, DroneID, MaintenanceDate, MaintenanceType, Description, MaintenanceStatus)
VALUES
(1, 'DRN001', '2026-07-01', 'Routine Inspection', 'Battery and motor inspection', 'Completed'),
(2, 'DRN002', '2026-07-02', 'Battery Check', 'Battery performance checked', 'Completed'),
(3, 'DRN003', '2026-07-03', 'Motor Service', 'Motor performance checked', 'Completed'),
(4, 'DRN004', '2026-07-04', 'Software Update', 'Flight software updated', 'Completed'),
(5, 'DRN005', '2026-07-05', 'Motor Repair', 'Motor maintenance required', 'In Progress'),
(6, 'DRN006', '2026-07-06', 'Routine Inspection', 'General inspection completed', 'Completed'),
(7, 'DRN007', '2026-07-07', 'Battery Check', 'Battery health checked', 'Completed'),
(8, 'DRN008', '2026-07-08', 'Propeller Replacement', 'Propellers replaced', 'Completed'),
(9, 'DRN009', '2026-07-09', 'Routine Inspection', 'General maintenance', 'Completed'),
(10, 'DRN010', '2026-07-10', 'Battery Repair', 'Battery connector repaired', 'Completed'),
(11, 'DRN011', '2026-07-11', 'Software Update', 'Navigation software updated', 'Completed'),
(12, 'DRN012', '2026-07-12', 'Motor Service', 'Motor performance checked', 'Completed'),
(13, 'DRN013', '2026-07-13', 'Routine Inspection', 'General inspection completed', 'Completed'),
(14, 'DRN014', '2026-07-14', 'Battery Check', 'Battery capacity checked', 'Completed'),
(15, 'DRN015', '2026-07-15', 'Motor Repair', 'Motor replacement required', 'In Progress'),
(16, 'DRN016', '2026-07-16', 'Software Update', 'System software updated', 'Completed'),
(17, 'DRN017', '2026-07-17', 'Routine Inspection', 'General inspection completed', 'Completed'),
(18, 'DRN018', '2026-07-18', 'Battery Check', 'Battery health checked', 'Completed'),
(19, 'DRN019', '2026-07-19', 'Propeller Replacement', 'Propellers replaced', 'Completed'),
(20, 'DRN020', '2026-07-20', 'Routine Inspection', 'General maintenance completed', 'Completed')
;
```

Messages

```
(20 row(s) affected)
```

Figure 18 : Insert Maintenance Records

```
INSERT INTO BatteryCharging
(ChargingID, DroneID, StartTime, EndTime, BatteryBefore, BatteryAfter)
VALUES
('CHG001', 'DRN001', '2026-08-01 06:00', '2026-08-01 07:30', 35.00, 95.00),
('CHG002', 'DRN002', '2026-08-02 06:30', '2026-08-02 08:00', 40.00, 96.00),
('CHG003', 'DRN003', '2026-08-03 07:00', '2026-08-03 08:30', 28.00, 90.00),
('CHG004', 'DRN004', '2026-08-04 06:00', '2026-08-04 07:20', 32.00, 94.00),
('CHG005', 'DRN005', '2026-08-05 05:30', '2026-08-05 07:00', 25.00, 92.00),
('CHG006', 'DRN006', '2026-08-06 06:00', '2026-08-06 07:30', 30.00, 93.00),
('CHG007', 'DRN007', '2026-08-07 06:30', '2026-08-07 08:00', 42.00, 98.00),
('CHG008', 'DRN008', '2026-08-08 07:00', '2026-08-08 08:20', 29.00, 91.00),
('CHG009', 'DRN009', '2026-08-09 06:00', '2026-08-09 07:30', 34.00, 94.00),
('CHG010', 'DRN010', '2026-08-10 05:30', '2026-08-10 07:00', 20.00, 90.00),
('CHG011', 'DRN011', '2026-08-11 06:00', '2026-08-11 07:20', 38.00, 95.00),
('CHG012', 'DRN012', '2026-08-12 06:30', '2026-08-12 08:00', 27.00, 92.00),
('CHG013', 'DRN013', '2026-08-13 06:00', '2026-08-13 07:30', 40.00, 96.00),
('CHG014', 'DRN014', '2026-08-14 07:00', '2026-08-14 08:30', 31.00, 93.00),
('CHG015', 'DRN015', '2026-08-15 05:30', '2026-08-15 07:00', 22.00, 89.00),
('CHG016', 'DRN016', '2026-08-16 06:00', '2026-08-16 07:20', 36.00, 94.00),
('CHG017', 'DRN017', '2026-08-17 06:30', '2026-08-17 08:00', 29.00, 91.00),
('CHG018', 'DRN018', '2026-08-18 06:00', '2026-08-18 07:30', 45.00, 98.00),
('CHG019', 'DRN019', '2026-08-19 05:30', '2026-08-19 07:00', 24.00, 90.00),
('CHG020', 'DRN020', '2026-08-19 06:00', '2026-08-19 07:30', 37.00, 95.00)
;
```

Messages

```
(20 row(s) affected)
```

Figure 19 : Insert BatteryCharging Records

```sql
-- FETCHING DATA FROM A TABLE --

SELECT * FROM Customer;
```

Results | Messages

	CustomerID	FirstName	LastName	Email	Phone	Address
1	1	Kamal	Perera	kamal.perera@gmail.com	0771234567	Colombo 03
2	2	Nimal	Fernando	nimal.fernando@gmail.com	0712345678	Kandy
3	3	Saman	Silva	saman.silva@gmail.com	0753456789	Galle
4	4	Amal	Perera	amal.perera@gmail.com	0764567890	Negombo
5	5	Kasun	Fernando	kasun.fernando@gmail.com	0785678901	Gampaha
6	6	Tharindu	Silva	tharindu.silva@gmail.com	0701234567	Matara
7	7	Ravindu	Perera	ravindu.perera@gmail.com	0772345678	Kurunegala
8	8	Chamod	Fernando	chamod.fernando@gmail.com	0713456789	Colombo 07
9	9	Dinesh	Silva	dinesh.silva@gmail.com	0754567890	Kalutara
10	10	Isuru	Perera	isuru.perera@gmail.com	0765678901	Panadura
11	11	Akila	Fernando	akila.fernando@gmail.com	0786789012	Wattala
12	12	Pasindu	Silva	pasindu.silva@gmail.com	0707890123	Kotte
13	13	Shehan	Perera	shehan.perera@gmail.com	0778901234	Dehiwala
14	14	Rashmika	Fernando	rashmika.fernando@gmail.c...	0719012345	Moratuwa
15	15	Janith	Silva	janith.silva@gmail.com	0750123456	Mount La...
16	16	Sachin	Perera	sachin.perera@gmail.com	0761234567	Rajagiriya
17	17	Vishwa	Fernando	vishwa.fernando@gmail.com	0782345678	Nugegoda
18	18	Dinuka	Silva	dinuka.silva@gmail.com	0703456789	Maharaga...
19	19	Yasiru	Perera	yasiru.perera@gmail.com	0774567890	Battaramu...
20	20	Malith	Fernando	malith.fernando@gmail.com	0715678901	Colombo 05

Figure 20 : Fetching Customer Data

```sql
SELECT * FROM DeliveryStatus;
```

Results | Messages

	StatusID	StatusName	Description
1	ST001	Pending	Delivery request has been received and is waiting ...
2	ST002	Confirmed	Delivery request has been confirmed by the system.
3	ST003	Assigned	A drone has been assigned to the delivery.
4	ST004	Pickup Scheduled	Pickup has been scheduled for the delivery.
5	ST005	Ready for Pickup	Parcel is ready to be collected from the pickup loc...
6	ST006	Picked Up	Parcel has been collected from the pickup location.
7	ST007	In Transit	Parcel is currently being transported to the destinat...
8	ST008	Arrived at Hub	Parcel has arrived at the distribution hub.
9	ST009	Processing	Parcel is being processed at the distribution hub.
10	ST010	Out for Delivery	Parcel is currently on the way to the drop-off locati...
11	ST011	Delivered	Parcel has been successfully delivered to the cust...
12	ST012	Delivery Failed	The delivery could not be completed successfully.
13	ST013	Cancelled	The delivery request has been cancelled.
14	ST014	Rescheduled	The delivery has been scheduled for another time.
15	ST015	Delayed	The delivery has been delayed due to operational ...
16	ST016	Returned	The parcel has been returned to the sender.
17	ST017	On Hold	The delivery is temporarily placed on hold.
18	ST018	Drone Assigned	A suitable drone has been selected for the delivery.
19	ST019	Pickup Completed	The parcel pickup process has been completed.
20	ST020	Completed	The complete delivery process has been successf...

Figure 21 : Fetching DeliveryStatus Data

```
SELECT * FROM DistributionHub;
```

Results | Messages

	HubID	HubName	Location	ContactNumber	Email
1	HUB001	Colombo Central Hub	Colombo 01	0112345678	colombo@distributionhub.lk
2	HUB002	Kandy Distribution Hub	Kandy	0812345678	kandy@distributionhub.lk
3	HUB003	Galle Distribution Hub	Galle	0912345678	galle@distributionhub.lk
4	HUB004	Gampaha Distribution Hub	Gampaha	0332345678	gampaha@distributionhub.lk
5	HUB005	Negombo Distribution Hub	Negombo	0312345678	negombo@distributionhub.lk
6	HUB006	Matara Distribution Hub	Matara	0412345678	matara@distributionhub.lk
7	HUB007	Kurunegala Distribution Hub	Kurunegala	0372345678	kurunegala@distributionhub.lk
8	HUB008	Kalutara Distribution Hub	Kalutara	0342345678	kalutara@distributionhub.lk
9	HUB009	Ratnapura Distribution Hub	Ratnapura	0452345678	ratnapura@distributionhub.lk
10	HUB010	Anuradhapura Distribution Hub	Anuradhapura	0252345678	anuradhapura@distributionhub.lk
11	HUB011	Jaffna Distribution Hub	Jaffna	0212345678	jaffna@distributionhub.lk
12	HUB012	Batticaloa Distribution Hub	Batticaloa	0652345678	batticaloa@distributionhub.lk
13	HUB013	Trincomalee Distribution Hub	Trincomalee	0262345678	trincomalee@distributionhub.lk
14	HUB014	Badulla Distribution Hub	Badulla	0552345678	badulla@distributionhub.lk
15	HUB015	Nuwara Eliya Distribution Hub	Nuwara Eliya	0522345678	nuwaraeliya@distributionhub.lk
16	HUB016	Avissawella Distribution Hub	Avissawella	0362345678	avissawella@distributionhub.lk
17	HUB017	Panadura Distribution Hub	Panadura	0382345678	panadura@distributionhub.lk
18	HUB018	Moratuwa Distribution Hub	Moratuwa	0113456789	moratuwa@distributionhub.lk
19	HUB019	Wattala Distribution Hub	Wattala	0114567890	wattala@distributionhub.lk
20	HUB020	Maharagama Distribution Hub	Maharagama	0115678901	maharagama@distributionhub.lk

Figure 22 : Fetching DistributionHub Data

```
SELECT * FROM Drone;
```

Results | Messages

	DroneID	HubID	DroneModel	BatteryLevel	PayloadCapacity	AvailabilityStatus
1	DRN001	HUB001	Sky-X1	95.50	9.00	Available
2	DRN002	HUB002	Sky-X2	88.20	9.00	Available
3	DRN003	HUB003	Sky-Pro1	76.80	7.50	In Use
4	DRN004	HUB005	Sky-Pro2	92.40	2.70	Available
5	DRN005	HUB005	Sky-X3	65.30	9.00	Maintenance
6	DRN006	HUB003	Sky-Max1	81.60	7.50	Available
7	DRN007	HUB004	Sky-Max2	97.10	9.00	Available
8	DRN008	HUB014	Sky-X4	72.50	2.70	In Use
9	DRN009	HUB008	Sky-Pro3	84.90	9.00	Available
10	DRN010	HUB007	Sky-Max3	59.40	7.50	Charging
11	DRN011	HUB008	Sky-X5	90.30	9.00	Available
12	DRN012	HUB009	Sky-M30 Series	68.70	2.70	In Use
13	DRN013	HUB010	Sky-Pro4	93.80	9.00	Available
14	DRN014	HUB012	Sky-Max4	79.20	7.50	Available
15	DRN015	HUB014	Sky-X1	61.50	9.00	Maintenance
16	DRN016	HUB001	Sky-M30 Series	86.40	2.70	Available
17	DRN017	HUB002	Sky-Max5	74.60	9.00	In Use
18	DRN018	HUB018	Sky-Max5	96.20	7.50	Available
19	DRN019	HUB020	Sky-X7	70.80	9.00	Charging
20	DRN020	HUB005	Sky-Pro6	89.50	2.70	Available

Figure 23 : Fetching Drone Data

```
SELECT * FROM Delivery;
```

Results | Messages

	DeliveryID	CustomerID	StatusID	PickupLocation	DropoffLocation	PickupTime	DropoffTime	ParcelWeight
1	DL001	1	ST011	Colombo 03	Colombo 01	2026-08-01 09:00:00.000	2026-08-01 10:30:00.000	2.50
2	DL002	2	ST007	Kandy	Peradeniya	2026-08-02 10:00:00.000	NULL	3.20
3	DL003	3	ST011	Galle	Unawatuna	2026-08-03 09:30:00.000	2026-08-03 11:20:00.000	1.80
4	DL004	4	ST006	Negombo	Katunayake	2026-08-04 11:00:00.000	NULL	4.50
5	DL005	5	ST007	Gampaha	Kadawatha	2026-08-05 09:00:00.000	NULL	5.00
6	DL006	6	ST011	Matara	Weligama	2026-08-06 08:30:00.000	2026-08-06 10:10:00.000	2.20
7	DL007	7	ST010	Kurunegala	Kuliyapitiya	2026-08-07 10:00:00.000	NULL	6.00
8	DL008	8	ST011	Colombo 07	Borella	2026-08-08 08:45:00.000	2026-08-08 10:00:00.000	1.50
9	DL009	9	ST007	Kalutara	Beruwala	2026-08-09 10:30:00.000	NULL	3.70
10	DL010	10	ST011	Panadura	Wadduwa	2026-08-10 08:00:00.000	2026-08-10 09:15:00.000	2.00
11	DL011	11	ST003	Wattala	Colombo 04	NULL	NULL	4.20
12	DL012	12	ST004	Kotte	Nugegoda	NULL	NULL	2.80
13	DL013	13	ST011	Dehiwala	Wellawatte	2026-08-13 08:45:00.000	2026-08-13 10:20:00.000	3.00
14	DL014	14	ST007	Moratuwa	Ratmalana	2026-08-14 10:00:00.000	NULL	4.80
15	DL015	15	ST011	Mount Lavinia	Dehiwala	2026-08-15 08:30:00.000	2026-08-15 09:45:00.000	1.20
16	DL016	16	ST005	Rajagiriya	Battaramulla	NULL	NULL	3.50
17	DL017	17	ST010	Nugegoda	Maharagama	2026-08-17 09:15:00.000	NULL	5.50
18	DL018	18	ST011	Maharagama	Kottawa	2026-08-18 08:15:00.000	2026-08-18 09:40:00.000	2.40
19	DL019	19	ST002	Battaramulla	Kotte	NULL	NULL	4.00
20	DL020	20	ST003	Colombo 05	Colombo 02	NULL	NULL	3.30

Figure 24 : Fetching Delivery Data

```
SELECT * FROM DroneAssignment;
```

Results | Messages

	AssignmentID	DeliveryID	DroneID	AssignmentDate	AssignedBy
1	1	DL001	DRN001	2026-08-01	Admin
2	2	DL002	DRN003	2026-08-02	Admin
3	3	DL003	DRN006	2026-08-03	System
4	4	DL004	DRN009	2026-08-04	Admin
5	5	DL005	DRN007	2026-08-05	Manager
6	6	DL006	DRN018	2026-08-06	System
7	7	DL007	DRN008	2026-08-07	Manager
8	8	DL008	DRN011	2026-08-08	Admin
9	9	DL009	DRN013	2026-08-09	System
10	10	DL010	DRN016	2026-08-10	Admin
11	11	DL011	DRN002	2026-08-11	Manager
12	12	DL012	DRN004	2026-08-12	System
13	13	DL013	DRN014	2026-08-13	Admin
14	14	DL014	DRN017	2026-08-14	Manager
15	15	DL015	DRN020	2026-08-15	System
16	16	DL016	DRN001	2026-08-16	Admin
17	17	DL017	DRN003	2026-08-17	Manager
18	18	DL018	DRN006	2026-08-18	System
19	19	DL019	DRN009	2026-08-19	Admin
20	20	DL020	DRN012	2026-08-19	Manager

Figure 25 : Fetching DroneAssignment Data

```
SELECT * FROM Maintenance;
```

Results | Messages

	MaintenanceID	DroneID	MaintenanceDate	MaintenanceType	Description	MaintenanceStatus
1	1	DRN001	2026-07-01	Routine Inspection	Battery and motor inspection	Completed
2	2	DRN002	2026-07-02	Battery Check	Battery performance checked	Completed
3	3	DRN003	2026-07-03	Motor Service	Motor performance checked	Completed
4	4	DRN004	2026-07-04	Software Update	Flight software updated	Completed
5	5	DRN005	2026-07-05	Motor Repair	Motor maintenance required	In Progress
6	6	DRN006	2026-07-06	Routine Inspection	General inspection completed	Completed
7	7	DRN007	2026-07-07	Battery Check	Battery health checked	Completed
8	8	DRN008	2026-07-08	Propeller Replacement	Propellers replaced	Completed
9	9	DRN009	2026-07-09	Routine Inspection	General maintenance	Completed
10	10	DRN010	2026-07-10	Battery Repair	Battery connector repaired	Completed
11	11	DRN011	2026-07-11	Software Update	Navigation software updated	Completed
12	12	DRN012	2026-07-12	Motor Service	Motor performance checked	Completed
13	13	DRN013	2026-07-13	Routine Inspection	General inspection completed	Completed
14	14	DRN014	2026-07-14	Battery Check	Battery capacity checked	Completed
15	15	DRN015	2026-07-15	Motor Repair	Motor replacement required	In Progress
16	16	DRN016	2026-07-16	Software Update	System software updated	Completed
17	17	DRN017	2026-07-17	Routine Inspection	General inspection completed	Completed
18	18	DRN018	2026-07-18	Battery Check	Battery health checked	Completed
19	19	DRN019	2026-07-19	Propeller Replacement	Propellers replaced	Completed
20	20	DRN020	2026-07-20	Routine Inspection	General maintenance completed	Completed

Figure 26 : Fetching Maintenance Data

```
SELECT * FROM BatteryCharging;
```

Results | Messages

	ChargingID	DroneID	StartTime	EndTime	BatteryBefore	BatteryAfter
1	CHG001	DRN001	2026-08-01 06:00:00.000	2026-08-01 07:30:00.000	35.00	95.00
2	CHG002	DRN002	2026-08-02 06:30:00.000	2026-08-02 08:00:00.000	40.00	96.00
3	CHG003	DRN003	2026-08-03 07:00:00.000	2026-08-03 08:30:00.000	28.00	90.00
4	CHG004	DRN004	2026-08-04 06:00:00.000	2026-08-04 07:20:00.000	32.00	94.00
5	CHG005	DRN005	2026-08-05 05:30:00.000	2026-08-05 07:00:00.000	25.00	92.00
6	CHG006	DRN006	2026-08-06 06:00:00.000	2026-08-06 07:30:00.000	30.00	93.00
7	CHG007	DRN007	2026-08-07 06:30:00.000	2026-08-07 08:00:00.000	42.00	98.00
8	CHG008	DRN008	2026-08-08 07:00:00.000	2026-08-08 08:20:00.000	29.00	91.00
9	CHG009	DRN009	2026-08-09 06:00:00.000	2026-08-09 07:30:00.000	34.00	94.00
10	CHG010	DRN010	2026-08-10 05:30:00.000	2026-08-10 07:00:00.000	20.00	90.00
11	CHG011	DRN011	2026-08-11 06:00:00.000	2026-08-11 07:20:00.000	38.00	95.00
12	CHG012	DRN012	2026-08-12 06:30:00.000	2026-08-12 08:00:00.000	27.00	92.00
13	CHG013	DRN013	2026-08-13 06:00:00.000	2026-08-13 07:30:00.000	40.00	96.00
14	CHG014	DRN014	2026-08-14 07:00:00.000	2026-08-14 08:30:00.000	31.00	93.00
15	CHG015	DRN015	2026-08-15 05:30:00.000	2026-08-15 07:00:00.000	22.00	89.00
16	CHG016	DRN016	2026-08-16 06:00:00.000	2026-08-16 07:20:00.000	36.00	94.00
17	CHG017	DRN017	2026-08-17 06:30:00.000	2026-08-17 08:00:00.000	29.00	91.00
18	CHG018	DRN018	2026-08-18 06:00:00.000	2026-08-18 07:30:00.000	45.00	98.00
19	CHG019	DRN019	2026-08-19 05:30:00.000	2026-08-19 07:00:00.000	24.00	90.00
20	CHG020	DRN020	2026-08-19 06:00:00.000	2026-08-19 07:30:00.000	37.00	95.00

Figure 27 : Fetching BatteryCharging Data

```sql
-- STORED PROCEDURE --

CREATE PROCEDURE GetCustomerDeliveries
    @CustomerID INT
AS
BEGIN
    SELECT
        d.DeliveryID,
        d.CustomerID,
        CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
        d.PickupLocation,
        d.DropoffLocation,
        ds.StatusName,
        d.PickupTime,
        d.DropoffTime,
        d.ParcelWeight

    FROM Delivery d
    INNER JOIN Customer c
        ON d.CustomerID = c.CustomerID
    INNER JOIN DeliveryStatus ds
        ON d.StatusID = ds.StatusID
    WHERE d.CustomerID = @CustomerID;
END;
```

Messages

Command(s) completed successfully.

Figure 28 : Stored Procedure

```sql
-- TEST STORED PROCEDURE --

EXEC GetCustomerDeliveries
    @CustomerID = 1;
```

Results | Messages

	DeliveryID	CustomerID	CustomerName	PickupLocation	DropoffLocation	StatusName	PickupTime	DropoffTime	ParcelWeight
1	DL001	1	Kamal Perera	Colombo 03	Colombo 01	Delivered	2026-08-01 09:00:00.000	2026-08-01 10:30:00.000	2.50

Figure 29 : Test Stored Procedure

```sql
-- TRIGGER --

CREATE TABLE drone_status_log(
    log_id INT IDENTITY(1,1) PRIMARY KEY,
    drone_id VARCHAR(10),
    old_status VARCHAR(30),
    new_status VARCHAR(30),
    changed_at DATETIME DEFAULT GETDATE()
);
```

Messages

Command(s) completed successfully.

```
CREATE TRIGGER after_drone_status_update
ON Drone
AFTER UPDATE
AS
BEGIN
    INSERT INTO drone_status_log(drone_id, old_status, new_status)
    SELECT
        i.DroneID,
        d.AvailabilityStatus,
        i.AvailabilityStatus
    FROM inserted i
    INNER JOIN deleted d
        ON i.DroneID = d.DroneID
    WHERE i.AvailabilityStatus <> d.AvailabilityStatus;
END;
```

Messages

```
Command(s) completed successfully.
```

```
UPDATE Drone
SET AvailabilityStatus = 'Maintenance'
WHERE DroneID = 'DRN001';
```

Messages

```
(1 row(s) affected)

(1 row(s) affected)
```

```
SELECT *FROM drone_status_log;
```

Results | Messages

	log_id	drone_id	old_status	new_status	changed_at
1	1	DRN001	Available	Maintenance	2026-08-24 13:29:47.973

Figure 30 : Create Trigger

TASK 5 - SQL queries

1) List all drones with their current status and battery level

```sql
-- List all drones with their current status and battery level -

SELECT
    DroneID,
    AvailabilityStatus AS CurrentStatus,
    BatteryLevel

FROM Drone
;
```

Results | Messages

	DroneID	CurrentStatus	BatteryLevel
1	DRN001	Maintenance	95.50
2	DRN002	Available	88.20
3	DRN003	In Use	76.80
4	DRN004	Available	92.40
5	DRN005	Maintenance	65.30
6	DRN006	Available	81.60
7	DRN007	Available	97.10
8	DRN008	In Use	72.50
9	DRN009	Available	84.90
10	DRN010	Charging	59.40
11	DRN011	Available	90.30
12	DRN012	In Use	68.70
13	DRN013	Available	93.80
14	DRN014	Available	79.20
15	DRN015	Maintenance	61.50
16	DRN016	Available	86.40
17	DRN017	In Use	74.60
18	DRN018	Available	96.20
19	DRN019	Charging	70.80
20	DRN020	Available	89.50

Figure 31 : Query 1

2) Find all deliveries that are currently 'In Transit'

```
SELECT
    d.DeliveryID,
    d.CustomerID,
    d.PickupLocation,
    d.DropoffLocation,
    ds.StatusName

FROM Delivery d
INNER JOIN DeliveryStatus ds
    ON d.StatusID = ds.StatusID
WHERE ds.StatusName = 'In Transit'
;
```

Results | Messages

	DeliveryID	CustomerID	PickupLocation	DropoffLocation	StatusName
1	DL002	2	Kandy	Peradeniya	In Transit
2	DL005	5	Gampaha	Kadawatha	In Transit
3	DL009	9	Kalutara	Beruwala	In Transit
4	DL014	14	Moratuwa	Ratmalana	In Transit

Figure 32 : Query 2

3) Display deliveries with customer names and assigned drone IDs

```
-- Display deliveries with customer names and assigned drone IDs --

SELECT
    d.DeliveryID,
    CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
    da.DroneID,
    d.PickupLocation,
    d.DropoffLocation
FROM Delivery d
INNER JOIN Customer c
    ON d.CustomerID = c.CustomerID
INNER JOIN DroneAssignment da
    ON d.DeliveryID = da.DeliveryID
;
```

Results | Messages

	DeliveryID	CustomerName	DroneID	PickupLocation	DropoffLocation
1	DL001	Kamal Perera	DRN001	Colombo 03	Colombo 01
2	DL002	Nimal Fernando	DRN003	Kandy	Peradeniya
3	DL003	Saman Silva	DRN006	Galle	Unawatuna
4	DL004	Amal Perera	DRN009	Negombo	Katunayake
5	DL005	Kasun Fernando	DRN007	Gampaha	Kadawatha
6	DL006	Tharindu Silva	DRN018	Matara	Weligama
7	DL007	Ravindu Perera	DRN008	Kurunegala	Kuliyapitiya
8	DL008	Chamod Fernando	DRN011	Colombo 07	Borella
9	DL009	Dinesh Silva	DRN013	Kalutara	Beruwala
10	DL010	Isuru Perera	DRN016	Panadura	Wadduwa
11	DL011	Akila Fernando	DRN002	Wattala	Colombo 04
12	DL012	Pasindu Silva	DRN004	Kotte	Nugegoda
13	DL013	Shehan Perera	DRN014	Dehiwala	Wellawatte
14	DL014	Rashmika Fernando	DRN017	Moratuwa	Ratmalana
15	DL015	Janith Silva	DRN020	Mount Lavinia	Dehiwala
16	DL016	Sachin Perera	DRN001	Rajagiriya	Battaramulla
17	DL017	Vishwa Fernando	DRN003	Nugegoda	Maharagama
18	DL018	Dinuka Silva	DRN006	Maharagama	Kottawa
19	DL019	Yasiru Perera	DRN009	Battaramulla	Kotte
20	DL020	Malith Fernando	DRN012	Colombo 05	Colombo 02

Figure 33 : Query 3

4) Calculate total number of deliveries handled by each drone

```sql
-- Calculate total number of deliveries handled by each drone --

SELECT
    dr.DroneID,
    dr.DroneModel,
    COUNT(da.DeliveryID) AS TotalDeliveries
FROM Drone dr
LEFT JOIN DroneAssignment da
    ON dr.DroneID = da.DroneID
GROUP BY
    dr.DroneID,
    dr.DroneModel
ORDER BY
    TotalDeliveries DESC
;
```

Results | Messages

	DroneID	DroneModel	TotalDeliveries
1	DRN001	Sky-X1	2
2	DRN003	Sky-Pro1	2
3	DRN006	Sky-Max1	2
4	DRN009	Sky-Pro3	2
5	DRN007	Sky-Max2	1
6	DRN008	Sky-X4	1
7	DRN004	Sky-Pro2	1
8	DRN002	Sky-X2	1
9	DRN011	Sky-X5	1
10	DRN012	Sky-M30 Series	1
11	DRN013	Sky-Pro4	1
12	DRN014	Sky-Max4	1
13	DRN016	Sky-M30 Series	1
14	DRN017	Sky-Max5	1
15	DRN018	Sky-Max5	1
16	DRN020	Sky-Pro6	1
17	DRN019	Sky-X7	0
18	DRN015	Sky-X1	0
19	DRN005	Sky-X3	0
20	DRN010	Sky-Max3	0

Figure 34 : Query 4

5) Find the top 3 most efficient drones (least average delivery time)

```
-- Find the top 3 most efficient drones --

SELECT TOP 3
    da.DroneID,
    AVG (
        DATEDIFF
        (
            MINUTE,
            d.PickupTime,
            d.DropoffTime
        )
    )
AS AverageDeliveryTimeMinutes

FROM Delivery d
INNER JOIN DroneAssignment da
    ON d.DeliveryID = da.DeliveryID
WHERE d.PickupTime  IS NOT NULL
  AND d.DropoffTime IS NOT NULL
GROUP BY
    da.DroneID
ORDER BY
    AverageDeliveryTimeMinutes ASC
;
```

Results | Messages

	DroneID	AverageDeliveryTimeMinutes
1	DRN011	75
2	DRN016	75
3	DRN020	75

Figure 35 : Query 5

TASK 6 - Database Security, Backup and Recovery

Database security is very important for the SkyDeliver drone delivery system because the database stores important information about customers, delivery requests, drones, distribution hubs, maintenance records and delivery status. If unauthorized users access the database, they may change or delete important information. Therefore, suitable security, access control, backup and recovery strategies should be implemented to protect the SkyDeliver database.

Database Security

Access Control

Access control is one of the methods to ensure that only authorized users can access the database. The level of access has to be regulated depending on the user's responsibilities to avoid illegal access and alteration of significant data.

The SkyDeliver system can use the following database roles:

Role	Responsibilities	Access
Database Administrator	Manage the complete database, users and security	Full access
Operations Manager	Manage deliveries, drone assignments and hubs	SELECT, INSERT, UPDATE
Delivery Staff	View and update delivery information	SELECT, UPDATE
Maintenance Staff	Manage drone maintenance and charging records	SELECT, INSERT, UPDATE
Reporting User	Generate delivery and performance reports	SELECT only

Table 38 : Database Roles

For example, a Reporting User only needs to view information and generate reports. Therefore, this user should not have permission to change or delete database records. This follows the principle of least privilege, where users receive only the access required for their work.

User Authentication

Users should log in using a username and strong password before accessing the system. Passwords should not be stored as plain text. They should be stored using secure password hashing.

For administrator accounts, Multi-Factor Authentication (MFA) can provide an additional layer of security. User accounts should also be reviewed regularly, and accounts that are no longer required should be disabled or removed.

Data Protection

SkyDeliver should protect customer and operational data from unauthorized access. Data transferred between the mobile application and server should use secure communication such as HTTPS/TLS.

The database server should also be protected using a firewall, and only trusted applications and authorized users should be allowed to connect to it.

Input validation and parameterized queries should be used to reduce the risk of SQL injection attacks. Primary keys, foreign keys and other database constraints should also be used to maintain data integrity.

Database Backup Strategy

Regular backups are required because database data can be lost due to hardware failures, software errors, accidental deletion or cyberattacks.

The SkyDeliver database can use the following backup methods:

- Full Backup:
  A complete copy of the database can be created daily.
- Incremental Backup:
  Data that has been changed since the last backup set is preserved by incremental. Incremental is suitable for saving space and reducing the time of the backup process.
- Transaction Log Backup:
  Transaction logs can be backed up regularly to help recover recent database changes.

Backup copies should not be stored only on the main server. At least one copy should be stored in a separate secure location or cloud storage.

Backup Security

Backup files may include sensitive customer and business information. Thus, backups must be encrypted and only accessible to authorized database administrators.

The company can also adopt the 3-2-1 rule, which involves storing 3 copies of the data, saving them on 2 different mediums and keeping at least 1 copy offline.

Regular testing of the backups is also required to ensure that a restore is possible.

Database Recovery Strategy

A recovery strategy is required to restore the SkyDeliver database after a failure or data loss.If the main database server fails, the latest full backup can be restored first. Then the latest incremental backup and transaction log backups can be applied to recover recent changes.

For example, after a database failure, the company should be able to recover important information such as delivery requests, drone assignments and delivery statuses. Recovery testing should also be performed regularly to make sure the database can be restored successfully.

Disaster Recovery

SkyDeliver should have a disaster recovery plan for serious problems such as server failure, fire, natural disasters or major cyberattacks.

A backup database or cloud-based recovery system can be maintained so that the service can be restored if the main database becomes unavailable.

The recovery plan should include backup locations, recovery procedures, responsible staff members and regular recovery testing.

Backup and Recovery Schedule

Activity	Frequency	Purpose
Full Backup	Daily	Create a complete copy of the database
Incremental Backup	Every few hours	Store recent changes
Transaction Log Backup	Regular intervals	Recover recent transactions

Backup Verification	Weekly	Check that backups are usable
Recovery Testing	Monthly	Confirm that the database can be restored

Table 39 : Backup and Recovery Schedule

This schedule helps reduce the risk of permanent data loss and ensures that recent backup copies are available when required.

TASK 7 - Test Plan and Test Cases

Testing is an essential part of the SkyDeliver Drone Delivery Management System database development process. The objective of testing activities is to ensure that the database works correctly and consistently stores all the data, providing the right results for different inputs and operations.

The testing process focuses on validating database tables, relationships, constraints, SQL queries, stored procedures, and data retrieval operations. Both positive and negative test cases are used to identify whether the system behaves as expected.

Test Plan

Test Objectives

The testing of the SkyDeliver database is carried out to ensure that the system works correctly and handles data safely. The main objectives are:

- To check whether all database tables are created correctly.
- To confirm that valid data can be added successfully.
- To test primary and foreign key constraints.
- To check that required fields do not allow invalid NULL values.
- To ensure SQL queries provide correct results.
- To verify the operation of the stored procedure.
- To check the accuracy of drone and delivery information.
- To evaluate delivery performance results.
- To find and fix errors before finalizing the database.

Test Scope

The testing covers the main database components of the SkyDeliver system.

- In Scope,

The following areas will be tested:

1) Customer table
2) Delivery table
3) Delivery Status table
4) Distribution Hub table
5) Drone table
6) Drone Assignment table
7) Maintenance table
8) Battery Charging table
9) Primary key constraints
10) Foreign key constraints
11) NOT NULL constraints
12) SQL queries
13) Stored procedures
14) Data retrieval
15) Delivery and drone relationships

- Out of Scope

The following areas are outside the scope of this database testing:

1) Mobile application user interface
2) Actual drone hardware
3) Physical drone flight operations
4) GPS hardware testing
5) Real-world delivery operations
6) Payment gateway integration

Testing Methods

Several testing methods will be used to validate the SkyDeliver database.

1. Functional Testing

Functional testing verifies whether the database performs the required operations correctly, such as inserting, updating, retrieving, and deleting data.

2. Constraint Testing

Primary key, foreign key, NOT NULL, UNIQUE, and other constraints will be tested to ensure that invalid data cannot be stored.

3. Query Testing

The SQL queries required by the assignment will be executed using the test data. The returned results will be compared with the expected results.

4. Negative Testing

Invalid inputs will be entered to verify that the database correctly rejects incorrect data.

For example, an invalid CustomerID can be used when inserting a delivery to test the foreign key constraint.

5. Integration Testing

Relationships between tables will be tested using SQL JOIN operations to verify that related records are retrieved correctly.

6. Stored Procedure Testing

The stored procedure implemented in the database will be executed with valid parameters and its output will be compared with the expected result.

Test Resources

The following resources are required to perform the testing.

Resource	Purpose
SQL Server	Database management and testing
SQL Server Management Studio (SSMS)	Execute SQL scripts and queries
SkyDeliver Database	System under test
Sample Test Data	Validate database operations
SQL Scripts	Create tables, constraints and database functions
Computer	Execute and document testing
Screenshots	Provide evidence of test results

Table 40 : Test Resources

Test Schedule

The testing activities can be carried out according to the following schedule.

Testing Activity	Duration
Database/table testing	Day 1
Sample data testing	Day 1
Constraint testing	Day 2
Relationship testing	Day 2
SQL query testing	Day 3
Stored procedure testing	Day 3
Error/negative testing	Day 4

Final verification	Day 4
Screenshot/documentation	Day 5

Table 41 : Test Schedule

Test Cases

The following test cases are designed to validate the SkyDeliver database.

Test ID	Test Case	Description	Expected Output	Actual Result	Status
TC001	Insert Valid Customer	Verify that a valid customer record can be inserted into the Customer table	The new customer record should be inserted successfully	The customer record was inserted successfully	PASS

```
--Test Cases--

INSERT INTO Customer
(CustomerID, FirstName, LastName, Email, Phone, Address)
VALUES
(21, 'Test', 'Customer', 'test.customer@gmail.com', '0771151032', 'Colombo');

SELECT *FROM Customer
WHERE CustomerID = 21;
```

```
Messages

(1 row(s) affected)
```

```
SELECT *FROM Customer
WHERE CustomerID = 21;
```

Results | Messages

	CustomerID	FirstName	LastName	Email	Phone	Address
1	21	Test	Customer	test.customer@gmail.com	0771151032	Colombo

Figure 36 : TC001

TC002	Duplicate Primary Key	Verify that the Customer table does not allow duplicate CustomerID values	The database should reject the record because CustomerID 1 already exists	The database rejected the record and displayed a primary key violation	PASS



Figure 37 : TC002

TC003	Invalid Foreign Key	Verify that an invalid CustomerID cannot be inserted into the Delivery table	The database should reject the record because CustomerID 999 does not exist in the Customer table	The database rejected the record due to the foreign key constraint	PASS

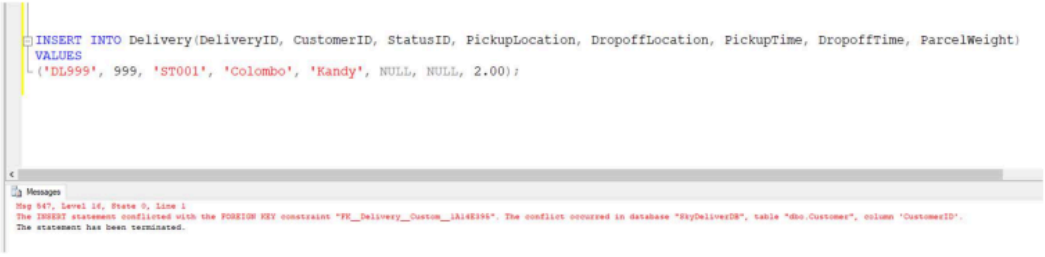


Figure 38 : TC003

TC004	NOT NULL Validation	Verify that required fields cannot contain NULL values	The database should reject the record because FirstName is a required field	The database rejected the record because of the NOT NULL constraint	PASS



Figure 39 : TC004

TC005	Find In Transit Deliveries	Verify that the database correctly identifies deliveries that are currently in transit	Only deliveries with the status In Transit should be displayed	The query returned the deliveries with the In Transit status	PASS

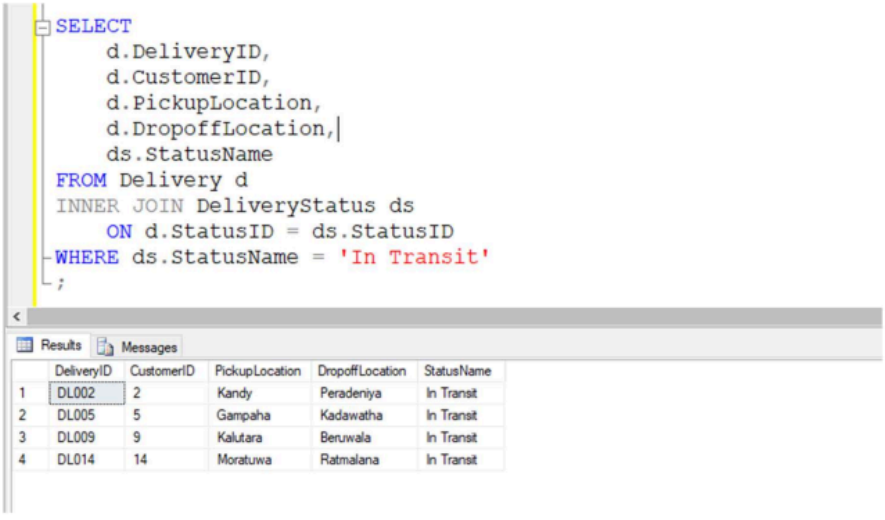


Figure 40 : TC005

TC006	Display Customer Names and Assigned Drone IDs	Verify that delivery information can be combined with customer names and assigned drone IDs	The query should display each delivery with the relevant customer name and assigned DroneID	Customer names and assigned DroneIDs were displayed correctly	PASS

```
SELECT
    d.DeliveryID,
    CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
    da.DroneID,
    d.PickupLocation,
    d.DropoffLocation
FROM Delivery d
INNER JOIN Customer c
    ON d.CustomerID = c.CustomerID
INNER JOIN DroneAssignment da
    ON d.DeliveryID = da.DeliveryID;
```

Results | Messages

	DeliveryID	CustomerName	DroneID	PickupLocation	DropoffLocation
1	DL001	Kamal Perera	DRN001	Colombo 03	Colombo 01
2	DL002	Nimal Fernando	DRN003	Kandy	Peradeniya
3	DL003	Saman Silva	DRN006	Galle	Unawatuna
4	DL004	Amal Perera	DRN009	Negombo	Katunayake
5	DL005	Kasun Fernando	DRN007	Gampaha	Kadawatha
6	DL006	Tharindu Silva	DRN018	Matara	Weligama
7	DL007	Ravindu Perera	DRN008	Kurunegala	Kuliyapitiya
8	DL008	Chamod Fernando	DRN011	Colombo 07	Borella
9	DL009	Dinesh Silva	DRN013	Kalutara	Beruwala
10	DL010	Isuru Perera	DRN016	Panadura	Wadduwa
11	DL011	Akila Fernando	DRN002	Wattala	Colombo 04
12	DL012	Pasindu Silva	DRN004	Kotte	Nugegoda
13	DL013	Shehan Perera	DRN014	Dehiwala	Wellawatte
14	DL014	Rashmika Fernando	DRN017	Moratuwa	Ratmalana
15	DL015	Janith Silva	DRN020	Mount Lavinia	Dehiwala
16	DL016	Sachin Perera	DRN001	Rajagiriya	Battaramulla
17	DL017	Vishwa Fernando	DRN003	Nugegoda	Maharagama
18	DL018	Dinuka Silva	DRN006	Maharagama	Kottawa
19	DL019	Yasiru Perera	DRN009	Battaramulla	Kotte
20	DL020	Malith Fernando	DRN012	Colombo 05	Colombo 02

Figure 41 : TC006

TC007	Stored Procedure Testing	Verify that the stored procedure correctly retrieves deliveries belonging to a selected customer	Only the deliveries belonging to CustomerID 1 should be displayed	The stored procedure returned the delivery records belonging to CustomerID 1	PASS

```
EXEC GetCustomerDeliveries
    @CustomerID = 1;
```

Results | Messages

	DeliveryID	CustomerID	CustomerName	PickupLocation	DropoffLocation	StatusName	PickupTime	DropoffTime	ParcelWeight
1	DL001	1	Kamal Perera	Colombo 03	Colombo 01	Delivered	2026-08-01 09:00:00.000	2026-08-01 10:30:00.000	2.50

Figure 42 : TC007

TC008	Top 3 Most Efficient Drones	Verify that the database can identify the three most efficient drones based on average delivery time	The three drones with the lowest average delivery time should be displayed	The query returned the three drones with the lowest average delivery time	PASS

```
SELECT TOP 3
    da.DroneID,
    AVG(
        DATEDIFF(
            MINUTE,
            d.PickupTime,
            d.DropoffTime
        )
    ) AS AverageDeliveryTimeMinutes
FROM Delivery d
INNER JOIN DroneAssignment da
    ON d.DeliveryID = da.DeliveryID
WHERE d.PickupTime IS NOT NULL
  AND d.DropoffTime IS NOT NULL
GROUP BY da.DroneID
ORDER BY AverageDeliveryTimeMinutes ASC
;
```

Results | Messages

	DroneID	AverageDeliveryTimeMinutes
1	DRN011	75
2	DRN016	75
3	DRN020	75

Figure 43 : TC008

TC009	List All Drones with Status and Battery	Verify that the database can display all drones with their current status and battery level	All drones should be displayed with their DroneID, current status, and battery level	The query displayed all drones with their current status and battery level	PASS

```
SELECT
    DroneID,
    AvailabilityStatus AS CurrentStatus,
    BatteryLevel
FROM Drone;
```

Results | Messages

	DroneID	CurrentStatus	BatteryLevel
1	DRN001	Available	95.50
2	DRN002	Available	88.20
3	DRN003	In Use	76.80
4	DRN004	Available	92.40
5	DRN005	Maintenance	65.30
6	DRN006	Available	81.60
7	DRN007	Available	97.10
8	DRN008	In Use	72.50
9	DRN009	Available	84.90
10	DRN010	Charging	59.40
11	DRN011	Available	90.30
12	DRN012	In Use	68.70
13	DRN013	Available	93.80
14	DRN014	Available	79.20
15	DRN015	Maintenance	61.50
16	DRN016	Available	86.40
17	DRN017	In Use	74.60
18	DRN018	Available	96.20
19	DRN019	Charging	70.80
20	DRN020	Available	89.50

Figure 44 : TC009

TC010	Find In-Transit Deliveries	Verify that the database can identify deliveries that are currently in transit	Only deliveries with the 'In Transit' status should be displayed	The query returned only deliveries with the 'In Transit' status	PASS

```
SELECT
    d.DeliveryID,
    d.CustomerID,
    d.PickupLocation,
    d.DropoffLocation,
    ds.StatusName
FROM Delivery d
INNER JOIN DeliveryStatus ds
    ON d.StatusID = ds.StatusID
WHERE ds.StatusName = 'In Transit';
```

Results | Messages

	DeliveryID	CustomerID	PickupLocation	DropoffLocation	StatusName
1	DL002	2	Kandy	Peradeniya	In Transit
2	DL005	5	Gampaha	Kadawatha	In Transit
3	DL009	9	Kalutara	Beruwala	In Transit
4	DL014	14	Moratuwa	Ratmalana	In Transit

Figure 45 : TC 010

TC011	Display Customer and Assigned Drone Details	Verify that the database can display delivery details with customer names and assigned drone IDs	Delivery ID, customer name, assigned drone ID, pickup location, and drop-off location should be displayed	The query successfully displayed delivery details with customer names and assigned drone IDs	PASS

```
SELECT
    d.DeliveryID,
    CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
    da.DroneID,
    d.PickupLocation,
    d.DropoffLocation
FROM Delivery d
INNER JOIN Customer c
    ON d.CustomerID = c.CustomerID
INNER JOIN DroneAssignment da
    ON d.DeliveryID = da.DeliveryID;
```

Results | Messages

	DeliveryID	CustomerName	DroneID	PickupLocation	DropoffLocation
1	DL001	Kamal Perera	DRN001	Colombo 03	Colombo 01
2	DL002	Nimal Fernando	DRN003	Kandy	Peradeniya
3	DL003	Saman Silva	DRN006	Galle	Unawatuna
4	DL004	Amal Perera	DRN009	Negombo	Katunayake
5	DL005	Kasun Fernando	DRN007	Gampaha	Kadawatha
6	DL006	Tharindu Silva	DRN018	Matara	Weligama
7	DL007	Ravindu Perera	DRN008	Kurunegala	Kuliyapitiya
8	DL008	Chamod Fernando	DRN011	Colombo 07	Borella
9	DL009	Dinesh Silva	DRN013	Kalutara	Beruwala
10	DL010	Isuru Perera	DRN016	Panadura	Wadduwa
11	DL011	Akila Fernando	DRN002	Wattala	Colombo 04
12	DL012	Pasindu Silva	DRN004	Kotte	Nugegoda
13	DL013	Shehan Perera	DRN014	Dehiwala	Wellawatte
14	DL014	Rashmika Fernando	DRN017	Moratuwa	Ratmalana
15	DL015	Janith Silva	DRN020	Mount Lavinia	Dehiwala
16	DL016	Sachin Perera	DRN001	Rajagiriya	Battaramulla
17	DL017	Vishwa Fernando	DRN003	Nugegoda	Maharagama
18	DL018	Dinuka Silva	DRN006	Maharagama	Kottawa
19	DL019	Yasiru Perera	DRN009	Battaramulla	Kotte
20	DL020	Malith Fernando	DRN012	Colombo 05	Colombo 02

Figure 46 : TC 011

TC012	Calculate Total Deliveries per Drone	Verify that the database can calculate the total number of deliveries handled by each drone	Each drone should be displayed with its total number of assigned deliveries in descending order	The query successfully calculated and displayed the total deliveries handled by each drone	PASS

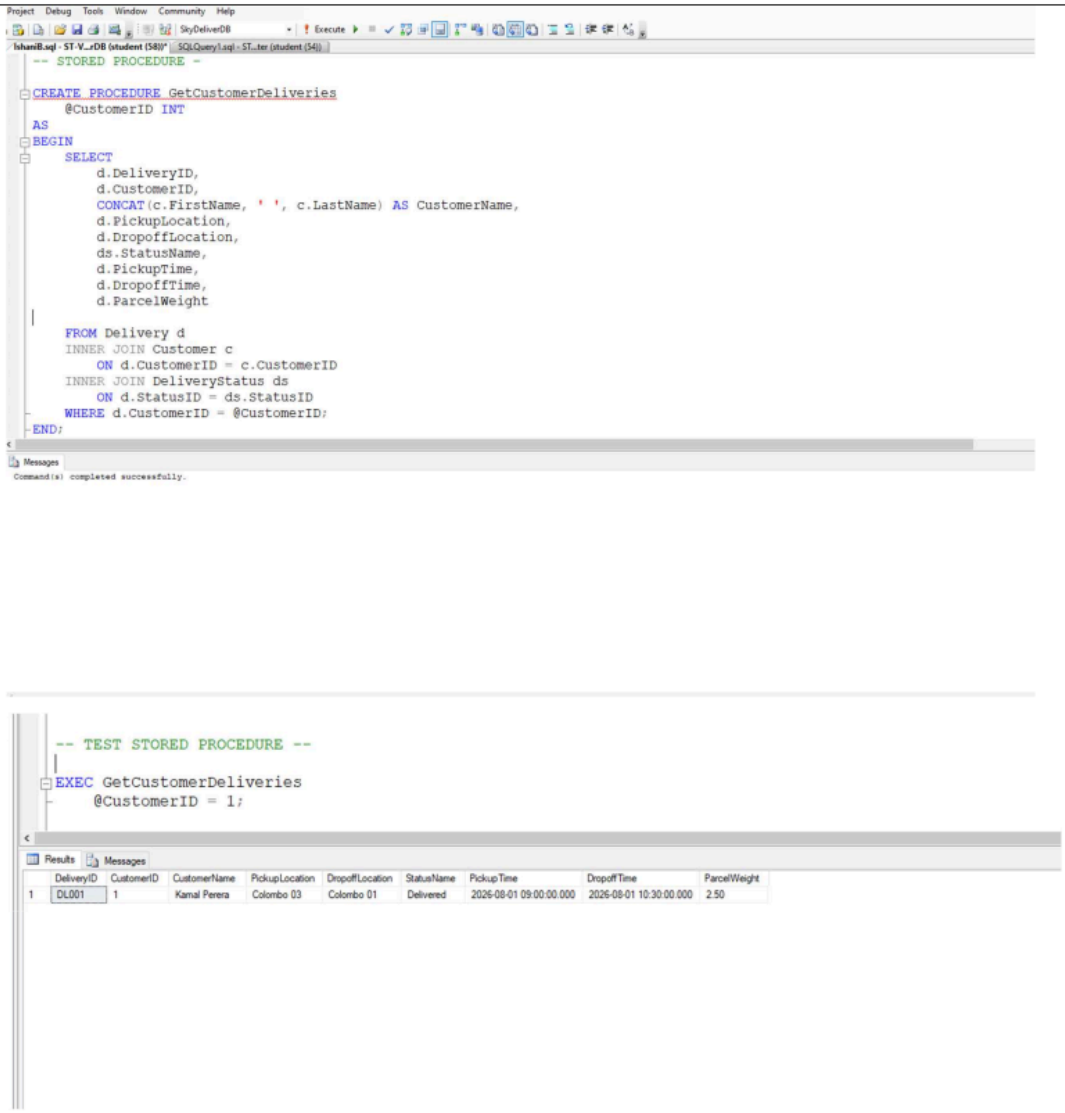
```
SELECT
    dr.DroneID,
    dr.DroneModel,
    COUNT(da.DeliveryID) AS TotalDeliveries
FROM Drone dr
LEFT JOIN DroneAssignment da
    ON dr.DroneID = da.DroneID
GROUP BY
    dr.DroneID,
    dr.DroneModel
ORDER BY
    TotalDeliveries DESC;
```

Results | Messages

	DroneID	DroneModel	TotalDeliveries
1	DRN001	Sky-X1	2
2	DRN003	Sky-Pro1	2
3	DRN006	Sky-Max1	2
4	DRN009	Sky-Pro3	2
5	DRN007	Sky-Max2	1
6	DRN008	Sky-X4	1
7	DRN004	Sky-Pro2	1
8	DRN002	Sky-X2	1
9	DRN011	Sky-X5	1
10	DRN012	Sky-M30 Series	1
11	DRN013	Sky-Pro4	1
12	DRN014	Sky-Max4	1
13	DRN016	Sky-M30 Series	1
14	DRN017	Sky-Max5	1
15	DRN018	Sky-Max5	1
16	DRN020	Sky-Pro6	1
17	DRN019	Sky-X7	0
18	DRN015	Sky-X1	0
19	DRN005	Sky-X3	0
20	DRN010	Sky-Max3	0

Figure 47 : TC 012

TC013	Get Customer Delivery Details	Verify that the stored procedure can retrieve delivery details for a selected customer	All deliveries belonging to the specified CustomerID should be displayed with customer name and delivery status	The stored procedure successfully returned the delivery details for the selected customer	PASS



```
-- STORED PROCEDURE --

CREATE PROCEDURE GetCustomerDeliveries
    @CustomerID INT
AS
BEGIN
    SELECT
        d.DeliveryID,
        d.CustomerID,
        CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
        d.PickupLocation,
        d.DropoffLocation,
        ds.StatusName,
        d.PickupTime,
        d.DropoffTime,
        d.ParcelWeight

    FROM Delivery d
    INNER JOIN Customer c
        ON d.CustomerID = c.CustomerID
    INNER JOIN DeliveryStatus ds
        ON d.StatusID = ds.StatusID
    WHERE d.CustomerID = @CustomerID;
END;
```

Messages

Command(s) completed successfully.

```
-- TEST STORED PROCEDURE --

EXEC GetCustomerDeliveries
    @CustomerID = 1;
```

Results | Messages

	DeliveryID	CustomerID	CustomerName	PickupLocation	DropoffLocation	StatusName	PickupTime	DropoffTime	ParcelWeight
1	DL001	1	Kamal Perera	Colombo 03	Colombo 01	Delivered	2026-08-01 09:00:00.000	2026-08-01 10:30:00.000	2.50

Figure 48 : TC 013

TC014	Log Drone Status Changes	Verify that the trigger records changes to a drone's availability status	The drone ID, old status, new status, and change time should be recorded in the status log	The trigger successfully recorded the drone status change in the drone_status_log table	PASS

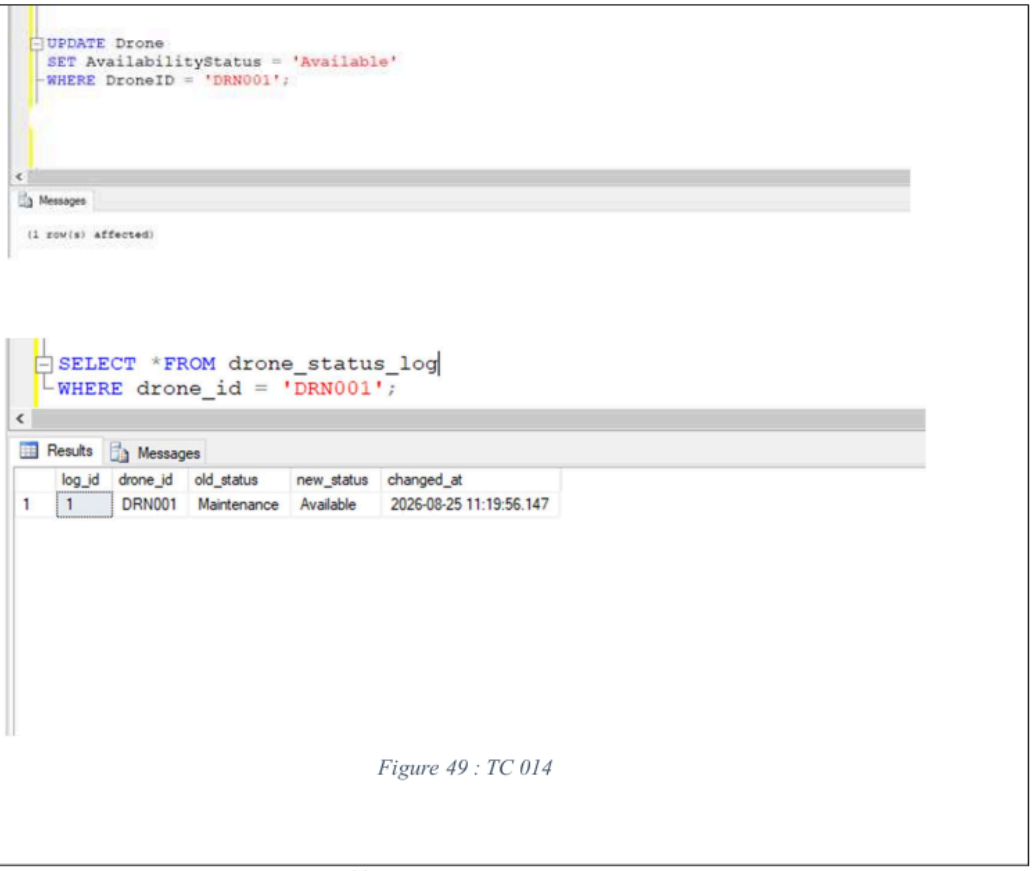


Figure 49 : TC 014

Table 42 : Test Cases

Conclusion

The SkyDeliver database was developed to support the main activities of a drone delivery service, including customers, deliveries, drones, hubs, assignments, maintenance, and battery charging.

A relational database was chosen because the system contains structured data and clear relationships. The ERD, primary keys, foreign keys, and 3NF normalization help maintain data accuracy and reduce unnecessary duplication.

The database was implemented using SQL Server with tables, constraints, sample data, and SQL queries. Testing was carried out to check the database operations and relationships. Security, access control, backup, and recovery were also considered to protect the data.

Overall, the database provides a reliable foundation for SkyDeliver. In the future, features such as GPS tracking, automated drone selection, demand prediction, and mobile application support could be added.

References

- Connolly, T. and Begg, C. (2015) *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th edn. Harlow: Pearson.
- Coronel, C. and Morris, S. (2019) *Database Systems: Design, Implementation, & Management*. 13th edn. Boston: Cengage Learning.
- Date, C.J. (2019) *Database Design and Relational Theory: Normal Forms and All That Jazz*. 2nd edn. Berkeley, CA: Apress.
- Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems*. 7th edn. Boston: Pearson.
- Garcia-Molina, H., Ullman, J.D. and Widom, J. (2009) *Database Systems: The Complete Book*. 2nd edn. Upper Saddle River, NJ: Pearson.
- Hernandez, M.J. (2013) *Database Design for Mere Mortals: A Hands-On Guide to Relational Database Design*. 3rd edn. Boston, MA: Addison-Wesley.
- IBM (2026) *What is a relational database?* Available at: IBM relational database overview (Accessed: 17 August 2026).
- ISO/IEC (2011) *ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models*. Geneva: International Organization for Standardization.
- Microsoft (2026) *SQL Server documentation*. Available at: Microsoft SQL Server documentation (Accessed: 17 August 2026).
- Microsoft (2026) *SQL Server security*. Available at: SQL Server security documentation (Accessed: 17 August 2026).
- Microsoft (2026) *Transact-SQL (T-SQL) documentation*. Available at: Transact-SQL documentation (Accessed: 17 August 2026).
- Microsoft (2026) *Backup and restore SQL Server databases*. Available at: Microsoft SQL Server backup and restore documentation (Accessed: 17 August 2026).
- MongoDB (2026) *MongoDB Documentation – Documents*. Available at: MongoDB documentation (Accessed: 17 August 2026).
- Myers, G.J., Sandler, C. and Badgett, T. (2011) *The Art of Software Testing*. 3rd edn. Hoboken, NJ: John Wiley & Sons.

- Oracle (2026) *Database Concepts*. Available at: Oracle Database Documentation (Accessed: 17 August 2026).
- Ramakrishnan, R. and Gehrke, J. (2003) *Database Management Systems*. 3rd edn. New York: McGraw-Hill.
- Silberschatz, A., Korth, H.F. and Sudarshan, S. (2019) *Database System Concepts*. 7th edn. New York: McGraw-Hill Education.

Buddhini Ishani Database.docx

ORIGINALITY REPORT

18%	9%	5%	15%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to University of Wales Institute, Cardiff Student Paper	4%
2	Submitted to Amsterdam hogeschool voor de kunsten Student Paper	2%
3	Submitted to Islington College,Nepal Student Paper	1%
4	Submitted to Kaplan College Student Paper	1%
5	www.coursehero.com Internet Source	1%
6	Submitted to Kingston University Student Paper	1%
7	Submitted to CINEC Campus Student Paper	1%
8	vignan.ac.in Internet Source	<1%
9	Submitted to The Robert Gordon University Student Paper	<1%
10	localedxcelcambridgeictcomputerclass.blogspot.com Internet Source	<1%
11	www.researchsquare.com Internet Source	<1%
12	Submitted to Asia Pacific University College of Technology and Innovation (UCTI) Student Paper	<1%
13	Submitted to Brunel University Student Paper	<1%
14	tsecurity.de Internet Source	<1%
15	Submitted to NCC Education Student Paper	<1%

16	Submitted to Manipal University Jaipur Online Student Paper	<1%
17	Submitted to Southern New Hampshire University - Continuing Education Student Paper	<1%
18	en.wikisource.org Internet Source	<1%
19	Submitted to Coventry University Student Paper	<1%
20	Submitted to SSBAS Student Paper	<1%
21	Submitted to School of Accounting & Management Student Paper	<1%
22	Submitted to University of Iowa Student Paper	<1%
23	Submitted to University of Hertfordshire Student Paper	<1%
24	Submitted to University of Maryland, Global Campus Student Paper	<1%
25	Submitted to University of Technology, Sydney Student Paper	<1%
26	business.custercountychief.com Internet Source	<1%
27	mefmobile.org Internet Source	<1%
28	Submitted to CTI Education Group Student Paper	<1%
29	desklib.com Internet Source	<1%
30	Antonios Lyronis, Emmanouil A. Varouchakis, Ioanna Anyfanti, George P. Karatzas. "A Unified Open-Source Metabase Design for Hydro-meteorological Data Descriptive Analysis", Environmental Processes, 2025 Publication	<1%
31	etd.aau.edu.et Internet Source	<1%
32	dspace-jcboseust.refread.com	

	Internet Source	<1%
33	jacksonlumber.cld.bz Internet Source	<1%
34	www.sweetstudy.com Internet Source	<1%
35	binus.ac.id Internet Source	<1%
36	www.lpude.in Internet Source	<1%
37	Elvis C. Foster, Shripad V. Godbole. "Database Systems - A Pragmatic Approach", CRC Press, 2022 Publication	<1%

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		